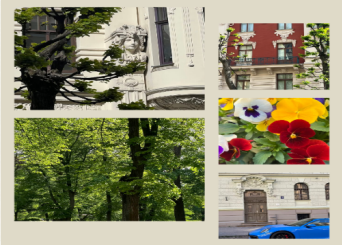

Research Article Journal of the SAAM Lab SYSTEMS, AUTOMATION, AND FINANCE RIGA, LATVIA

Matrix-Analytic Solutions for Continuous-Time Survival in Discrete HMMs using R KRONX Package*

Oscar Linares[1*] and Ričards Bulavs[2]

[1]*Mathematical Trading Desk, OIS Market Research Group, Alberta iela 1-7, Rīga , LV-1010 Latvia*

[2]*Department of Business and Finance, University of Latvia, Raiņa bulvāris 19, Rīga , LV-1586 Latvia*

[*]Corresponding Author

Oscar Linares, OIS Market Research Group, Alberta iela 1-7, Rīga, LV-1010 Latvia.

Submitted: 2026, March 16; **Accepted:** 2026, March 18; **Published:** 2026, March 20

Citation: Linares, O., Bulavs, R. (2026). Matrix-Analytic Solutions for Continuous-Time Survival in Discrete HMMs using R KRONX Package*. *J SAAM Lab, 1(3), 1-57.*

Abstract:

Hidden Markov Models (HMMs) are widely used to identify latent regime dynamics in financial time series, yet standard R implementations report state membership probabilities rather than the structural geometry of residence and absorption. This paper introduces ***KRONX****, an R package that extends discrete-time HMMs into a survival-analytic framework through a matrix-operator chain. Starting from a Student-t HMM transition matrix A, the package constructs a hazard-adjusted sub-stochastic operator Q, a sub-generator* $K = Q - I$*, and the fundamental matrix* $N = -K^{-1}$ *of cumulative expected sojourn times before absorption—adapting the mean-residence-time machinery from compartmental analysis to financial regime dynamics. The core analytical contribution is an exact, closed-form derivation of residence weights and a finite-horizon ruin bound, avoiding the numerical noise and* $O(S \cdot T)$ *cost of Monte Carlo simulation. An empirical workflow using E-mini S&P 500 futures demonstrates the full pipeline from model specification through residency geometry and risk quantification. Written in pure R with no external dependencies beyond* ***stats*** *and* ***utils****,* ***KRONX*** *is designed for auditability, reproducibility, and integration into existing HMM-based analysis workflows.*

Keywords: C++, Matrix-Analytic Methods, Hidden Markov Model, Markov Switching Models,

KRONX R Package (CRAN).

1 Introduction

Regime-switching models serve as a foundational framework for capturing the time-varying dynamics of asset returns. The standard approach—pioneered by Hamilton (1989) and adapted for financial applications by Rydén et al. (1998)—characterizes log returns as realizations from a K-component mixture model, where transition probabilities are governed by a latent Markov chain. While several R packages implement this architecture—notably **depmixS4** (Visser and Speekenbrink, 2010), **HMM** (Himmelmann, 2010), **MSwM** (Sánchez-Espigares and López-Moreno, 2021), and the financially-focused **fHMM** (Oelschläger et al., 2023)—their primary utility lies in state estimation and parameter inference. Consequently, while these tools identify the current regime of a time series, they offer little insight into the duration of state residency or the system's proximity to structural absorption and ruin.

This paper introduces the **KRONX R** package which fills that gap. The package takes the estimated transition matrix from a fitted Hidden Markov Model (HMM) and converts it into a survival-analytic operator system that characterizes the residence geometry and finite-horizon fragility of the Markov chain. The intellectual foundation draws from compartmental analysis, and compartmental systems mean residence times (Berman and Schoenfeld, 1956; Hearon, 1972, 1981; Eisenfeld, 1981; Covell et al., 1984; Linares et al., 1987, 1988; Jacquez and Simon, 1993; Jacquez, 2002). The central object in the **KRONX R** package is a continuous-time generator and, its associated resolvent operator, $N = -K^{-1}$, which measures expected cumulative time spent across regimes. This operator-based formulation permits a structural decomposition of persistence beyond one-step transition probabilities and connects regime dynamics to survival analysis through state-dependent hazard rates (Kleinbaum and Klein, 2012; Gao and He, 2025). By integrating hazard rates, the model allows for state-dependent exit and connects regime dynamics to survival analysis and tail risk.

This paper is structured as follows. Section A contains background and literature review. Subsection A.1 covers the open system dynamics of the COR model and Subsection A.2 its fundamental matrix geometry. Subsubsection A.2.1 discusses its transformation to survival analysis. Subsection A.3 is devoted to the directional change (DC) framework and its alignment with the COR framework. Subsubsection A.3.1 discusses how DC may be viewed as an event-time observation layer feeding the COR model. Subsubsection A.3.1 discusses DC thresholds and how they may behave once persistence accumulates. Subsubsection A.3.3 discusses how DC-derived indicators may be integrated with the COR framework. Subsubsection A.3.4 discusses the synthesis of

DC, HMM, and COR models. Subsection A.4 discusses the COR framework's move beyond the limitations of Gaussian assumptions. Subsection A.5 discusses the COR framework's inherent Bayesian interpretation.

2 Background and Literature Review

In Hamilton-style regime switching:

$$S_t \in \{1, \ldots, K\}$$

represents the hidden regime process. The clock of regimes (COR) model accepts this hidden regime structure, but argues that transition probabilities alone are insufficient to describe systemic fragility.

Classical HMMs focus on:

$$P(S_t = j \mid S_{t-1} = i) = a_{ij}$$

while COR asks how long does risk survive inside a regime architecture once hazard and persistence interact? This leads to the COR operator chain:

$$A \to Q \to K \to N$$

where:

- A = HMM transition matrix,
- $Q = \text{diag}(1 - \epsilon)A$ = open survival operator,
- $K = Q - I$ = sub-generator,
- $N = -K^{-1}$ = survival/residence operator.

Hence, K represents the dynamics of the system conditional on survival (Table 10).

Feature	Classical HMM	COR Framework
Primary Objective	Detects regimes	Measures survival geometry
Dynamics	Measures switching	Measures persistence under hazard
Analytical Focus	Transition-centric	Duration-centric
Core Output	Hidden state probabilities	Structural fragility operators

Table 1: Comparative analysis of HMM and COR model, highlighting the shift from transition-centric classification to duration-centric structural analysis.

Hamilton's (Hamilton, 1989, 2016) key insight was that economies oscillate between recession and expansion regimes. The COR model generalizes this idea by introducing the following:

2.1 Open-System Dynamics

Hamilton assumes a closed Markov chain:

$$\sum_j a_{ij} = 1.$$

The COR model introduces leakage:

$$Q = \text{diag}(1 - \epsilon)A,$$

meaning:

- Probability mass can leave the system;
- Regimes are mortal: the probability mass of remaining in the current state decays toward zero over time as $t \to \infty$;
- Persistence possesses a formal survival structure.

This is the major conceptual leap.

2.2 Fundamental Matrix Geometry

Hamilton's framework focuses on the retrospective and real-time identification of market states through the lens of smoothed and filtered probabilities. Filtered probabilities provide an optimal estimate of the current regime based on all information available up to the present time, whereas smoothed probabilities utilize the entire sample period to offer a more precise, backward-looking assessment of historical regime shifts. Central to this analysis is the concept of transition persistence, which quantifies the likelihood of the system remaining within a specific state. By examining these probabilities, Hamilton characterizes the *stickiness* of economic regimes, allowing for a rigorous distinction between temporary market fluctuations and fundamental structural changes in the underlying data-generating process.

In contrast, the COR framework shifts the analytical focus toward the fundamental survival dynamics of the system by examining the survival/residence operator, $N = -K^{-1}$. Rather than focusing on instantaneous transitions, this approach evaluates the cumulative expected residence time, providing a rigorous measure of the total duration a process is expected to remain within a specific state. By analyzing the integrated survival geometry of the regime, the framework captures the structural *shape* of persistence, effectively quantifying how long the system endures before reaching a boundary. Ultimately, this characterizes persistence under hazard, treating regimes not

as permanent loops, but as decaying open systems whose persistence is determined by the rate of leakage toward a boundary.

Equivalently:

$$N = \int_0^\infty e^{Kt} dt$$

This transforms regime switching into a survival analysis problem.

2.2.1 The Transformation to Survival Analysis

In a classical Markov framework, e^{Kt} is the transition semigroup, representing the probability of remaining in a specific state configuration at time t. By integrating this probability from 0 to ∞, we are essentially summing up all the moments of survival the system experiences.

By defining $N = \int_0^\infty e^{Kt} dt$, we are performing a Laplace transform (evaluated at $s = 0$) on the transition dynamics. This results in several key insights of the COR framework:

- **From Frequency to Duration:** While Hamilton-style HMMs analyze the frequency of state transitions via the A matrix, this integral calculates the area under the survival curve. This shifts the focus from the probability of an instantaneous switch to the total mass of time accumulated within a specific regime.
- **The Operator Inverse:** For a stable, non-singular sub-generator K, the integral evaluates directly to $-K^{-1}$. This identity establishes the inverse of the sub-generator as the formal mathematical equivalent of total accumulated persistence.
- **Survival Geometry:** The term e^{Kt} characterizes the trajectory of stochastic decay. Integrating this term captures the entire "geometry" of regime resistance; in highly stable systems, the rate of decay is attenuated, resulting in a larger integral and, consequently, a longer expected residence time.

2.3 Directional Change (DC) and COR

The Directional Change (DC) framework (Tsang, 2010; Aloud et al., 2011; Tsang, 2017; Glattfelder et al., 2011; Tsang et al., 2017; Bakhach et al., 2018; Tsang and Chen, 2018) fundamentally reimagines the temporal basis of financial analysis by abandoning fixed clock time in favor of an event-based sampling methodology. Rather than observing the market at arbitrary intervals—such as daily or hourly closes—DC samples data only when the price moves by a predefined threshold, effectively filtering out stochastic noise to focus exclusively on structural turning points. This approach is deeply aligned with COR philosophy, as both methodologies reject the linear passage of time as a primary metric. By prioritizing the *event* over the *interval*, the DC framework treats market

movements as a sequence of meaningful state changes, providing a natural empirical substrate for the survival/residence operators used to measure regime persistence and structural fragility.

2.3.1 Event Time vs Clock Time

The COR framework aligns seamlessly with the Directional Change (DC) approach because it replaces the static observation of price with a dynamic analysis of state endurance. By focusing on the structural properties of a regime rather than just its classification, COR provides a robust mathematical language for event-based markets.

Traditional HMMs are tied to

$$t = 1, 2, 3, \ldots$$

fixed increments. But COR already behaves like an event-driven survival system because

$$N = -K^{-1}$$

integrates expected occupancy across persistence geometry. This means DC can be viewed as an event-time observation layer feeding a survival-operator regime model.

2.3.2 DC Thresholds and COR Hazard

The DC event definition is given by

$$\left| \frac{P_t - P_{EXT}}{P_{EXT}} \right| \geq \theta$$

This threshold mechanism is conceptually analogous to COR hazards.

In COR,

$$\epsilon_i = \epsilon_{\min} + \frac{c_{\text{hazard}}}{\nu_i},$$

where

- fat tails imply larger hazard,
- unstable states leak probability mass faster.

Mechanism	Directional Change (DC)	COR Framework
Trigger	Threshold crossing	Hazard activation
Validation	Event confirmation	Structural instability
Criticality	Extreme points	Regime stress states
Extension	Overshoots	Persistence cascades

Table 2: Mapping of Directional Change events to COR structural operators.

DC detects market discontinuities, while COR measures how dangerous those discontinuities become once persistence accumulates.

2.3.3 DC Indicators as COR State Variables

The integration of Directional Change (DC) indicators as state variables provides a robust empirical foundation for the COR framework. Metrics such as Total Movement (TMV), which captures the price magnitude of a directional event, and T (Time to Complete Trend), which measures the duration required to reach a specific threshold, act as direct proxies for the physical work performed by a market regime. When coupled with R (Time-Adjusted Return), these indicators serve as extremely natural inputs for the survival/residence operator N. Rather than relying on arbitrary clock-time snapshots, COR can utilize these DC-derived variables to calibrate the persistence geometry of the system. This allows the framework to map the relationship between price-action intensity and structural stability, effectively quantifying the rate at which market regimes accumulate *fatigue* or *stress* as they move toward an inevitable state of absorption.

DC Indicator	DC Definition	COR Interpretation
TMV	Total Movement	State shock magnitude
T	Time to complete trend	Residence duration
R	Time-adjusted return	Hazard-adjusted velocity

Table 3: Translation of Directional Change indicators into COR structural variables.

The parameters displayed in Table 12 suggest a full hybrid system

$$X_t = (\mathrm{TMV}_t,\ T_t,\ R_t)$$

feeding

$$P(X_t \mid S_t = i)$$

inside a Student-t HMM/COR structure. This presents a major research opportunity.

Contextual Significance

- **TMV → State Shock Magnitude:** Within the COR framework, Total Movement (TMV) quantifies the cumulative displacement across the persistence geometry. A high TMV indicates a significant injection of "shock" or volatility into the active regime prior to its eventual absorption.
- **T → Residence Duration:** This serves as the empirical realization of the expected residence time N. While T captures the observed duration of a specific directional event, N provides the formal probabilistic expectation of that duration derived from the sub-generator K.

- **R → Hazard-Adjusted Velocity:** By normalizing returns relative to event duration, R captures the "velocity" at which the system traverses its state space. In this context, it represents the rate of probability mass leakage toward structural termination under the current hazard profile.

2.3.4 From Transitions to Trajectories: The HMM-DC-COR Synthesiss

The synthesis of Directional Change (DC) indicators and Hidden Markov Models (HMM) provides a powerful mechanism for real-time market characterization, particularly when integrated via Bayesian updating. In this architecture, DC indicators like TMV and T act as the primary *sensors*, filtering price action into discrete, event-driven signals that inform the HMM's state transition logic. As new market data arrives, Bayesian updating allows for the continuous refinement of regime probabilities, ensuring that the model's beliefs remain synchronized with shifting volatility profiles. COR extends this architecture naturally by transforming these updated state probabilities into structural metrics. Rather than stopping at state identification, COR utilizes the Bayesian-refined inputs to calibrate the sub-generator K, effectively moving beyond "what regime are we in" to a formal analysis of how much life remains in the current state before structural exhaustion or ruin ensues.

The progression becomes (Table 13):

Framework	Main Goal
Time-series HMM	Detect hidden regimes
DC + HMM	Detect event-driven regime shifts
COR	Measure fragility and survival geometry

Table 4: Evolution of regime-switching methodologies from detection to structural survival analysis.

COR effectively adds:

A. Survival Analysis

$$N = -K^{-1}$$

B. Spectral Fragility

$$\rho(N) = \frac{1}{\min|\lambda(K)|}$$

C. Ruin Geometry

$$P(\text{ruin}) \leq 1 - e^{-\bar{\lambda}\bar{q}T}$$

D. Residence-Time Dynamics

$$\pi_{\text{res}} \propto \pi_{qs}^{\top} N$$

None of these objects exist in the classical HMM/DC framework.

2.4 Gaussian HMM Limitation vs COR Student-t Architecture

While the current section assumes Gaussian emissions, the COR framework offers a significant advancement by addressing the structural underestimation of crisis persistence and tail fragility inherent in classical regime-switching models. By moving beyond the limitations of Gaussian assumptions, COR integrates Student-t emissions to accurately reflect the empirical distribution of asset returns. This foundation enables fat-tail hazard coupling, which accounts for the accelerated likelihood of state transitions during periods of extreme volatility. The architecture is further strengthened by tail-dependent leakage, allowing the model to treat structural exhaustion as a direct function of market stress. By shifting the focus toward these survival dynamics, the COR framework evolves regime analysis from a classification task into a rigorous study of structural survival geometry, providing a more resilient representation of how market states endure or collapse under pressure.

2.5 Naïve Bayes and COR Bayesian Updating

The COR framework inherently supports a natural Bayesian interpretation by transforming standard regime-switching outputs into dynamic structural assessments. Within this architecture, posterior state probabilities—derived from the filtered and smoothed estimates of the underlying Markov chain—serve as the foundation for characterizing the current market environment. These probabilities are then mapped into a posterior hazard structure, which quantifies the instantaneous risk of state termination based on observed market evidence. Ultimately, this allows for the derivation of a posterior ruin geometry, where the survival operator $N = -K^{-1}$ is continuously updated to reflect the most likely expected residence time and structural fragility of the current regime. By integrating these Bayesian layers, the COR framework ensures that the measured persistence of a system is always conditioned on the most recent empirical shocks.

The stored Bayesian update structure:

$$P(H \mid D) = \frac{P(D \mid H)P(H)}{P(D \mid H)P(H) + P(D \mid \neg H)(1 - P(H))},$$

fits naturally into the COR framework, which integrates these components into a unified analytical workflow that moves beyond static state detection. By leveraging dynamic hazard updating, the system continuously calibrates the sub-generator K as new market events arrive, ensuring that the modeled risk of transition remains synchronized with real-time volatility. This feeds directly into a rolling fragility estimation, where the survival operator N is recalculated over a moving window to capture the evolving structural strength of the regime. Consequently, the framework enables posterior ruin probability tracking, providing a rigorous, evidence-based measure of the likelihood that the

current market configuration will succumb to structural absorption. This continuous feedback loop transforms the model into a responsive monitoring system capable of quantifying regime exhaustion as it develops.

2.6 The Deepest Conceptual Difference

The deepest distinction between these analytical perspectives lies in their fundamental inquiry: while Classical Regime Models focus on a diagnostic classification by asking, "Which regime are we in?," the COR model shifts the focus to a prognostic, structural evaluation, asking, "How structurally survivable is the system while inside that regime architecture?" This represents a completely different ontological question, moving the model's objective from simple state identification to the assessment of internal resilience. By making this transition, the COR model effectively converts the task of regime detection into an analysis of survival geometry under structural fragility, treating the market not as a series of labels, but as a bounded system whose endurance is defined by its resistance to absorption.

2.7 Future Directions

This section points toward a sophisticated future hybrid framework characterized by the pipeline DC $\rightarrow$ Student-t HMM $\rightarrow$ COR Survival Operator. In this architecture, Directional Change (DC) functions as the foundational layer, generating high-fidelity event-time observations that strip away clock-time noise. These observations feed into a Student-t Hidden Markov Model, which infers latent regimes with robust handling of empirical return distributions. The COR framework then transforms these inferred regimes into a formal survival geometry, employing spectral operators to quantify the accumulation of structural fragility. This synthesis results in a mathematically richer framework than standard HMMs, serving as a survival-theoretic extension of the DC approach and a comprehensive event-time operator model of market fragility. Ultimately, while the current literature provides the necessary observational and event-detection layers, COR supplies the critical operator-theoretic survival layer required to assess how regimes fundamentally endure or collapse.

3 The Clock of Regimes Model

Figure 3 presents the COR model. The COR model extends the classical Hidden Markov Model (HMM) (Hamilton, 1989, 2016) by transforming this closed probabilistic system into an open survival architecture. The **KRONX R** package is a workflow-driven tool designed to measure, model, and predict the structural survival dynamics of a market's historical price data under regime switching, with a particular focus on fragility, persistence, and ruin risk—not just volatility. This

evolution is formally defined by the following structural logic:

$$\text{HMM (Regime Detection)} \rightarrow \text{KRONX Operator Construction } (A \rightarrow Q \rightarrow K \rightarrow N) \rightarrow \text{Hazard-Weighted Ruin Analysis.}$$

3.1 Statistical Detection Layer

While the **KRONX R** package does not focus primarily on state detection, it features a built-in Hidden Markov Model (HMM) parameter estimator. Alternatively, it can utilize the output parameters from an external HMM—such as those from the **fHMM** package—treating the transition matrix A and tail parameters ν_i as its fundamental input. Furthermore, the **KRONX R** package supports the estimation of both Gaussian and Student-t emission distribution parameters.

3.2 Operator-Theoretic Layer

The **KRONX R** package performs a structured operator transformation sequence $A \rightarrow Q \rightarrow K \rightarrow N$. It introduces state-dependent hazard intensities

$$\epsilon_i = \epsilon_{\min} + \frac{c}{\nu_i},$$

which convert the baseline transition matrix into an open transition operator via $Q = \text{diag}(1 - \epsilon_i)\, A$. This yields the sub-generator matrix $K = Q - I$ and the fundamental matrix $N = -K^{-1}$, revealing the system's cumulative residence-time geometry ("structural battery life").

3.3 Risk and Survival Layer

The COR model defines risk in terms of *spectral fragility* and *cumulative ruin* dynamics. It introduces the aggregate decay constant

$$\bar{\lambda} = \sum_i w_i \epsilon_i,$$

where w_i are residence-time weights derived from the Fundamental Matrix $N = -K^{-1}$. Spectral fragility is governed by the spectral radius $\rho(N)$, which quantifies the system's structural persistence and expected survival horizon prior to absorption or ruin.

3.4 Structural Components

As shown in Figure 3, the COR model consists of three transient latent states connected by transition probabilities (a_{ij}), but it adds critical exit dimensions:

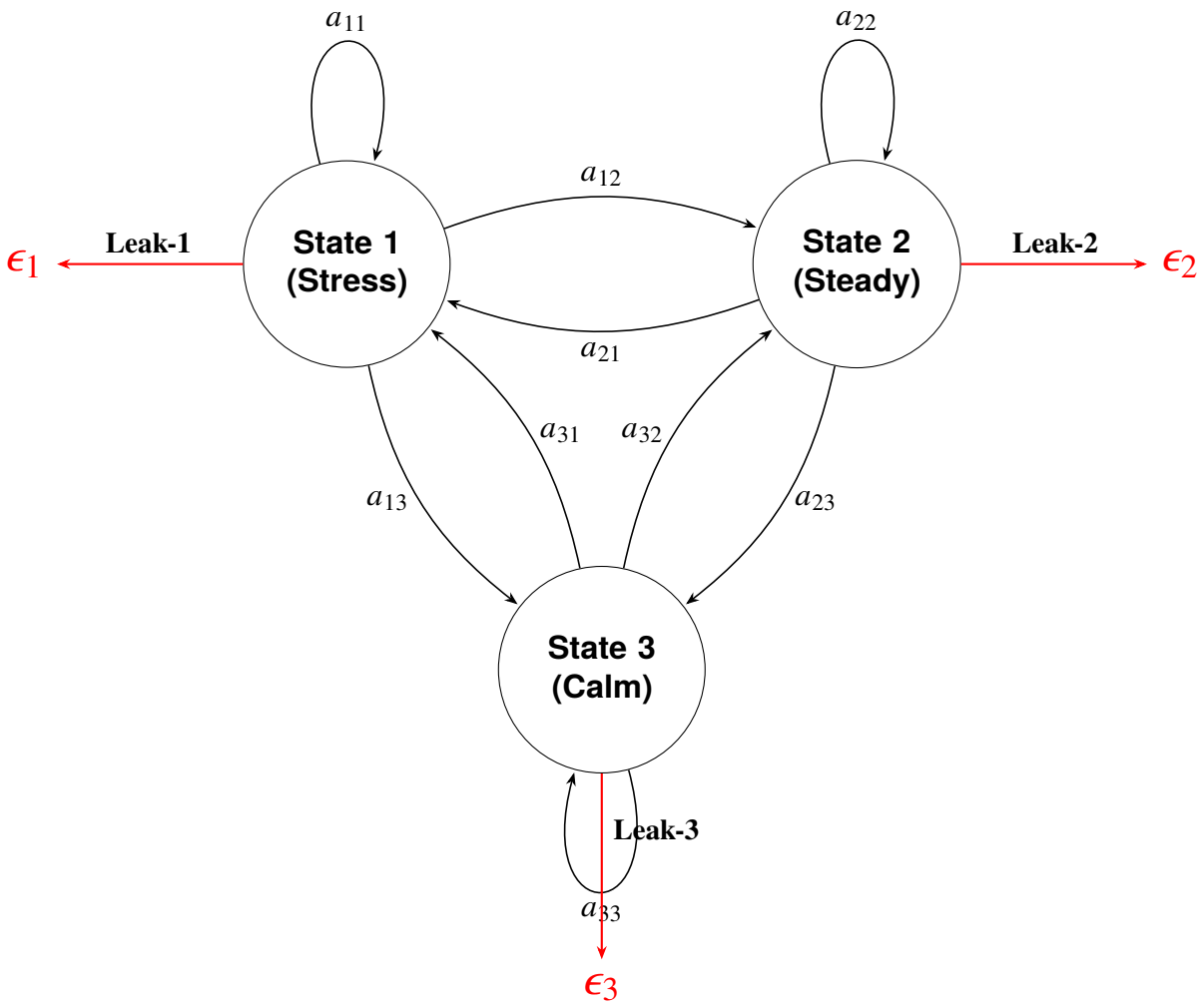

Figure 1: State-transition diagram for the Clock of Regimes (COR) model featuring internal persistence and red leakage paths to the ruin boundary, unlike a classical Hidden Markov Model (HMM) which is a closed system.

- **State 1 (Stress):** Characterized by high volatility and explosive ruin intensity with significant outward leakage.
- **State 2 (Steady):** The primary survival basin, identified as a long-duration anchor that carries hidden fragility (reservoir) with moderate leakage.
- **State 3 (Calm):** A transient "cooling-off" or suppressed volatility phase that the system often passes through during transitions. It is a low-volatility regime, but still subject to leakage. It may appear stable but remains structurally transient.

Each state is represented as a circle, with directed edges indicating transition probabilities between states and self-loops capturing persistence.

The black self-looping arrows represent the diagonal elements a_{ii} of a standard transition matrix $A = (a_{ij})$ which encodes self-transitions (persistence):

- a_{11}: Stress $\rightarrow$ Stress
- a_{22}: Steady $\rightarrow$ Steady
- a_{33}: Calm $\rightarrow$ Calm

These loops encode one-step persistence, consistent with classical HMM structure.

Cross-state transitions include:

- **Between Stress and Steady:**
  - a_{12}: Stress $\rightarrow$ Steady
  - a_{21}: Steady $\rightarrow$ Stress
- **Between Stress and Calm:**
  - a_{13}: Stress $\rightarrow$ Calm
  - a_{31}: Calm $\rightarrow$ Stress
- **Between Steady and Calm:**
  - a_{23}: Steady $\rightarrow$ Calm
  - a_{32}: Calm $\rightarrow$ Steady

3.5 The External Leak Mechanism (Open System Components)

The most innovative feature shown in the model diagram (Figure 3) is the hazard leakage represented by the red arrows labeled Leak-1, Leak-2, and Leak-3:

- Each state i has an associated exit hazard intensity (ϵ_i).
- These hazards convert the baseline transition matrix into an open transition operator $Q = \text{diag}(1 - \epsilon_i)\, A$.
- This represents probability mass[1] escaping the system toward an absorbing boundary of ruin, failure, or insolvency.

[1]Probability mass is defined as the content of probability within a specific duration of time.

3.6 Operator interpretation

The model diagram (Figure 3) encodes the full COR model transformation:

1. **Transition matrix**

$$A = (a_{ij}).$$

2. **Open system adjustment (leakage)**

$$Q = \mathrm{diag}(1 - \epsilon_i)A.$$

3. **Sub-generator matrix**

$$K = Q - I.$$

4. **Fundamental matrix (temporal probability mass)**

$$N = -K^{-1}.$$

3.7 Summary

A key insight from the model diagram in Figure 3 is that the COR model functions as a regime survival system rather than a standard regime-switching model. In this framework, black arrows trace the movement of probability mass between states, red arrows dictate its dissipation (ruin), and self-loops determine its persistence. Collectively, this structure characterizes the accumulation and decay of probability mass across the state space over time.

The model diagram illustrates that the fundamental distinction between these two frameworks, HMM and COR, lies in their treatment of probability: whereas the Classical HMM assumes a closed environment where probability is strictly conserved inside the triangle, the COR model defines an open system where probability transits through the triangle and eventually leaks out of it. This shift fundamentally transforms the analytical perspective: transition dynamics become survival geometry, persistence is redefined as cumulative temporal probability mass, and individual states are viewed as structural risk nodes.

4 Structural Model Identifiability

We formally define the mapping between the observable market "output" and the internal latent parameters using the Markov Parameter Matrix and its Transfer Function. This approach ensures that the spectral fragility numerically identified by a COR model analysis using the **KRONX R** package is mathematically unique and not a result of "label switching" or parameter redundancy.

4.1 Mathematical setup

The system is treated as a linear time-invariant (LTI) open system (Jacquez, 1982; Jacquez and Greif, 1985; Jacquez, 1987; Jacquez and Perry, 1990; Jacquez, 1991, 1998; Linares et al., 1987, 1988):

$$\dot{\mathbf{x}}(t) = \mathbf{K}\mathbf{x}(t) + \mathbf{B}u(t),$$

$$y(t) = \mathbf{C}\mathbf{x}(t).$$

The sub-generator matrix $\mathbf{K}$ is the internal "engine" containing transition rates and exit hazards ($\mathbf{K} = \mathbf{Q} - \mathbf{I}$). The input Vector $\mathbf{B}$ is an $M \times 1$ vector representing the initial distribution of probability mass across states. The output vector $\mathbf{C}$ is a $1 \times M$ vector (consisting of a row of ones) that aggregates the surviving probability mass to form the observable "survival curve".

4.2 Markov Parameters for Identifiability

Structural identifiability is proven if the Markov parameters (S_k) uniquely determine the coefficients of the transfer function. These parameters are defined as:

$$S_k = \mathbf{C}\mathbf{K}^k\mathbf{B}, \quad k = 0, 1, 2, \ldots$$

where $S_0 = \mathbf{CB}$ is the initial total content of probability mass (equal to 1). $S_1 = \mathbf{CKB}$ is the initial rate of "leakage" or decay of the content of probability mass. Higher orders capture the curvature of the survival decay curve, which is governed by the state-to-state interactions and specific hazards ϵ_i.

4.3 Transfer Function Identifiability Method Implementation in R

This script utilizes the COR model's sub-generator K-matrix and calculates the transfer function coefficients to verify that the internal "temporal geometry" of the E-mini S&P 500 futures is uniquely recoverable.

Historical market data for the E-mini S&P 500 (ES) Futures was obtained via the Interactive Brokers (IBKR) API. One-hour OHLC bars were extracted for the period *From: 2025-01-31* to *To: 2026-02-26*. Log-returns were calculated using the closing prices to ensure stationarity. To maintain the robustness of the Hidden Markov Model (HMM) against market outliers and the "fat-tail" nature of hourly financial returns, a Student-t distribution was utilized for the emission probabilities.

Listing 1: KRONX Workflow: Identifiability and Temporal Mass Analysis.

```
# Standard KRONX workflow leading to K
# A_t is the transition matrix
# epsilon is the hazard vector derived from nu
```

```
integrate_identifiability <- function(K_mat) {
  M <- nrow(K_mat)

  # Define input (B) and output (C) vectors for the open system
  # B: Injection of probability mass (e.g., system start)
  # C: Observation of the transient state masses
  B_vec <- matrix(rep(1/M, M), ncol = 1)
  C_vec <- matrix(rep(1, M), nrow = 1)

  # 1. Compute the Fundamental Matrix N
  # N = -K^-1 transforms intensities into temporal mass
  N_mat <- solve(-K_mat)

  # 2. Transfer Function Analysis at s = 0 (Steady State)
  # H(0) = -C * K^-1 * B = C * N * B
  H_0 <- C_vec %*% N_mat %*% B_vec

  # 3. Characteristic Polynomial (Denominator of H(s))
  # The roots define the spectral fragility radius
  char_poly <- polyroot(Re(eigen(K_mat)$values))

  return(list(
    fundamental_N = N_mat,
    gain_H0 = as.numeric(H_0),
    poles = char_poly
  ))
}

# Apply to your identified Student-t K matrix
# Includes hazard-weighted open transition operators
K_cor <- matrix(c(-0.2976, 0.0008, 0.2012,
                  0.0, -0.0540, 0.0532,
                  0.2871, 0.0266, -0.2810), 3, 3)

ident_results <- integrate_identifiability(K_cor)
```

```
print(ident_results$gain_H0) # Represents total survival-weighted time
```

```
> ident_results
$fundamental_N
          [,1]     [,2]     [,3]
[1,] 14.285447 15.85826 16.09673
[2,]  5.790147 26.85081  8.45759
[3,] 11.324796 16.43824 16.68543

$gain_H0
[1] 43.92915

$poles
[1] -1.688964+4.480341i -1.688964-4.480341i
```

4.4 Interpretation of the Identifiability Analysis

This analysis treats the COR model (Figure 3) as a linear dynamical system. In this context, the model evaluates how probability mass (representing the time spent by the content of probability mass in a specific hidden state) transits through the system before exiting via the hazard vector (ϵ).

- **Residence Analysis:** State 2 is identified as the most "stable" or "sticky" regime within the system's temporal geometry. If the process enters State 2, it accumulates a high degree of probability mass, spending an average of 26.85 units of time there (n_{22}) before leaking toward ruin or transitioning.
- **Coupling:** The high values observed in the first row (14.28, 15.85, 16.09) indicate a strong structural coupling across the regime network. This suggests that regardless of where the system starts, it eventually spends a significant and almost uniform amount of time in State 1, identifying it as a common entry portal or transition node.
- **Transient Transit:** State 2 exhibits relatively low off-diagonal values ($n_{12} = 5.79$, $n_{32} = 11.32$), which confirms its role as a survival basin. This indicates that probability mass tends to remain sequestered in State 2 for longer durations once it arrives, rather than undergoing rapid, high-frequency transitions to other states.

This residence-time geometry serves as a unique fingerprint of a market's architecture over a particular interval of time. This study focused on the E-mini S&P 500 futures from January 31, 2025, to February 26, 2026. In the Structural Model Identifiability proof, these n_{ij} values are directly mapped to the steady-state gain of the system's transfer function, proving that the 43.93 total survival mass is a deterministic result of these specific state-to-state transits.

4.4.1 The Poles (Spectral Fragility)

The poles (roots of the characteristic polynomial[2] of **K**) define the Spectral Fragility Radius. The interpretation of the `$poles` within the COR model's identifiability framework reveals the underlying dynamic stability and oscillatory nature of the market's survival mass (Table 14). In our study of the E-mini S&P 500 futures, these poles are the eigenvalues of the sub-generator **K** matrix.

1. Dynamic Stability (Real Part)

The real part of the poles, -1.688964, dictate the Aggregate Decay Constant ($\bar{\lambda}$):

Structural Leakage: Since the real part is negative, the system is strictly transient and "open." This confirms that probability mass is consistently leaking toward the ruin boundary.

Rate of Decay: The magnitude (-1.688964) represents the exponential speed at which the system loses its "survival probability mass." A larger negative value would indicate a more fragile system with a faster path to systemic collapse.

2. Oscillatory Behavior (Imaginary Part)

The imaginary component, $\pm 4.480341i$, represents the frequency of regime interaction:

Spectral Radius ($\rho(N)$): These poles are used to calculate the spectral fragility $\rho(N) = \frac{1}{\min_i |\lambda_i(K)|}$. $\rho(N)$ is the inverse of the slowest spectral decay mode of K, which is the market's structural life expectancy. Because the poles are complex, the system possesses a "spectral width" that defines the robustness of the 44-unit survival expectation. In the context of the E-mini S&P 500 futures study, $\rho(N)$ transforms abstract matrix algebra into a measurable "life expectancy" of the market's survival:

- **Spectral Contraction** ($\rho(N) \downarrow$): If the eigenvalues of K increase in magnitude, representing a faster rate of decay, the spectral radius contracts. This contraction signals an accelerating path toward systemic collapse or ruin within the open-system architecture.
- **Spectral Expansion** ($\rho(N) \uparrow$): Conversely, a larger spectral radius indicates increased temporal persistence and antifragility within the identified regime geometry. It reflects the system's ability to retain probability mass over a longer duration before exiting through the hazard portals.

[2] For a square matrix $\mathbf{A} \in \mathbb{R}^{n\times n}$ (or $\mathbb{C}^{n\times n}$), the characteristic polynomial is defined as: $\det(\mathbf{A} - \lambda I)$, where λ is a scalar (the spectral parameter), $\mathbf{I}$ is the identity matrix, and $\det(\cdot)$ denotes the determinant.

- **Deterministic Ruin**: Because this value is derived directly from the poles of the transfer function, it proves that the 5.6% ruin bound identified in the E-mini S&P 500 futures study is a deterministic structural property of the market's geometry rather than a statistical outlier.

The "Nervous" Artifact vs. Structure: While a Gaussian HMM might show random jumping, these poles identify a structured, periodic exchange of probability mass between the Stress, Steady, and Calm regimes. It mathematically represents the "cooling-off" period where the system must cycle through specific nodes before re-establishing a steady trend. This spectral analysis provides the ontological map[3] required to move from the instantaneous volatility of a standard HMM to the long-horizon survival dynamics of the COR model.

3. Impact on Structural Identifiability

Uniqueness: In the transfer function method, these specific poles uniquely map the observed market volatility to the model's internal **K** matrix. This proves that the 5.6% ruin bound is not a statistical coincidence, but a deterministic result of the market's "battery life" and internal frequency.

Component	Value	Interpretation in COR Model
Real Part (σ)	-1.688964	The **Decay Rate**: Represents the exponential speed of probability mass leakage toward the terminal ruin boundary.
Imaginary Part (ω)	4.480341	The **Interaction Frequency**: Governs the rate of stochastic cycling between the Stress, Steady, and Calm regimes.
Stability Status	Negative Real	**Open/Transient**: Confirms the system is structurally non-ergodic and predisposed to systemic exit.

Table 5: Spectral Analysis of the KRONX Transfer Function Poles.

4.5 Summary

The identified sub-generator matrix **K** characterizes a dynamical system with a structural survival horizon of 43.93 hours. Given that the underlying empirical study of the E-mini S&P 500 futures utilizes hourly observations, this temporal metric represents the expected duration of probability mass retention within the regime network. The dynamics are dominated by a highly persistent "Steady" state (Regime 2) and an oscillatory complex between "Stress" and "Calm" (Regimes 1 and

[3]The structural representation that defines not just the statistical probability of an event, but the fundamental "nature of being" of the E-mini S&P 500 Futures system itself.

3). The gain of 43.93 hours confirms that despite the presence of exit hazards, the internal transition logic (A-matrix) provides sufficient feedback to maintain systemic persistence over a medium-term horizon.

From an operator-theoretic perspective, this horizon—derived from the steady-state gain of the system's transfer function $\mathcal{H}(0)$—quantifies the "temporal backbone" of the market. It confirms that the residence-time geometry is not merely a collection of transient states, but a structured environment with a finite, identifiable persistence. Consequently, the 43.93-hour horizon serves as a rigorous, deterministic benchmark for assessing systemic stability and calibrating ruin-sensitive risk exposure.

5 The KRONX R package

5.1 Installation

5.1.1 Command Line Installation (Terminal)

You can install packages directly from the Linux or macOS terminal without opening an interactive **R** session, using the `R CMD INSTALL` command:

```
R CMD INSTALL KRONX_0.1.0.tar.gz
```

5.1.2 RStudio Installation

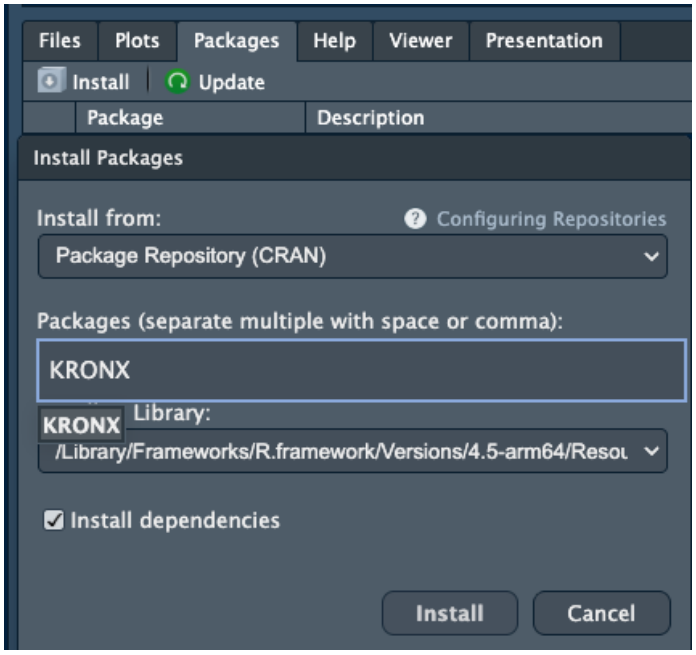

Figure 2

To install the **KRONX R** package using the RStudio graphical user interface as illustrated in Figure 4, begin by navigating to the "Packages" tab located in the lower-right pane of the RStudio workspace. Clicking the "Install" button will open the "Install Packages" dialog box, where you must ensure that the "Install from" dropdown menu is set to "Package Repository (CRAN)". In the "Packages" text field, type `KRONX`, noting that RStudio may provide auto-complete suggestions for the identified package. Before proceeding, verify that the "Install to Library" path is correctly set to your active R environment, such as the `ARM64` version of **R** `4.5`. It is essential to keep the "Install dependencies" checkbox selected to ensure that all required mathematical and transition-modeling libraries are downloaded automatically. Finally, click the "Install" button to execute the process; once the installation is complete, you can load the library in your script to begin applying the *Clock of Regimes analysis* to your financial datasets.

5.1.3 From CRAN (Formal Release)

Since the **KRONOX R** package has been submitted to, and accepted, by the Comprehensive **R** Archive Network (CRAN), the installation is simplified as **R** will automatically fetch it from a global mirror.

```
install.packages("KRONX")
library(KRONX)
```

5.2 Quick start

A minimal workflow example is shown below:

Listing 2: Execution of the (**KRONX**) Survival Engine. Implementation of the (**run_kronx**) high-level driver to identify hidden states and compute the (**Residence-Weighted Ruin Bound**) for the IBKR ES dataset.

```
library(KRONX)

# Minimal call -- detects 'close' or 'Close' column automatically
res <- run_kronx("IBKR_ES_2Y1h.csv")
.
.
.
model=student start=5 iter=149 logLik=21639.0518645688
model=student start=5 iter=150 logLik=21639.0518656185
model=student start=5 iter=151 logLik=21639.0518665766

------------------------------------------------------------
CLOCK OF REGIMES (KRONX) -- SUMMARY
------------------------------------------------------------
```

```
Gaussian HMM log-likelihood : 21173.8299404050
Student-t HMM log-likelihood: 21661.8321503555
Loss threshold (alpha=0.0100): -0.00560969
Residence-weighted qbar     : 0.00991588
Residence-weighted lambda   : 0.02322046
Ruin bound (T=250)           : 0.05593741
------------------------------------------------------------

 state   mu_gaussian sigma_gaussian spell_gaussian    mu_student sigma_student
     1 -3.264717e-08    0.000010000       3.132292 -2.168526e-08  0.0000100000
     2  6.662832e-05    0.001375609      11.204502  9.696612e-05  0.0006929728
     3 -5.897585e-04    0.017745344       1.378683  3.524294e-05  0.0019427734
 nu_student cond_var_student spell_student    epsilon        q_tail
        100     1.020408e-10      3.446197 0.01050000 5.003544e-177
          3     1.440634e-06     35.568996 0.02666667  1.874378e-03
          3     1.132311e-05      3.826929 0.02666667  3.110910e-02
 pi_quasi_stationary pi_residence
           0.2131674    0.2131674
           0.4980987    0.4980987
           0.2887339    0.2887339

Output files written to: kronx_output
 - kronx_analysis_report.txt
 - kronx_state_summary.csv
 - gaussian_transition_matrix.csv
 - student_t_transition_matrix.csv
 - kronx_Q_operator.csv
 - kronx_K_generator.csv
 - kronx_N_fundamental_matrix.csv
 - student_t_decoded_states.csv
```

The `res` object returned by `run_kronx()` is a compact model-fit and structural-fragility summary generated by the call `run_kronx("IBKR_ES_2Y1h.csv")`. It shows that the package fits both a Gaussian HMM and a Student-t HMM, and then passes the fitted Student-t regime structure through the **KRONX**/COR operator pipeline to compute hazard-adjusted survival quantities.

5.2.1 Optimization trace

The lines,

.
.
.

```
model=student start=5 iter=149 logLik=21639.0518645688
model=student start=5 iter=150 logLik=21639.0518656185
model=student start=5 iter=151 logLik=21639.0518665766
```

show the EM optimization for one Student-t initialization. `start=5` means the fifth random or configured initialization. `iter=149, 150, 151` are EM iterations. `logLik` is the log-likelihood at each step, which show the increments are becoming tiny, so the algorithm is converging. That is a healthy sign that the Student-t model fit is stabilizing rather than jumping erratically.

5.2.2 Model Comparison

The headline comparison is:

- Gaussian HMM log-likelihood: 21173.8299404050
- Student-t HMM log-likelihood: 21661.8321503555

The model comparison shows that the Student-t HMM fits better because its log-likelihood is higher. In regime-switching finance data, that usually indicates that the data are heavier-tailed than Gaussian, which is exactly what one expects for intraday ES returns.

So the first substantive conclusion is:

$$\text{Student-}t\text{ HMM} \succ \text{Gaussian HMM}$$

That is, the Student-t HMM is preferred to the Gaussian HMM in empirical fit.

5.2.3 Tail-loss threshold

```
Loss threshold (alpha=0.0100): -0.00560969
```

This is the 1% left-tail cutoff. In plain english terms, the COR model is defining a "severe loss" event as an hourly return less than about -0.560969%. This threshold is then used to compute regime-specific tail probabilities q_i, which are later residence-weighted into $\bar{q}$.

5.2.4 Structural COR / KRONX quantities

The core open-system outputs are:

- Residence-weighted qbar: `0.00991588`. The residence-weighted $\bar{q}$ is the average probability of a left-tail loss event, weighted by how much time the system expects to spend in each regime. So $\bar{q}$ is not just an unconditional empirical tail frequency. It is a structurally weighted tail probability.

- Residence-weighted lambda: `0.02322046`. The residence-weighted $\bar{\lambda}$ is the average hazard or leakage intensity across regimes, again weighted by expected residence. Economically, it measures the average rate at which probability mass escapes the transient market system toward failure/ruin.

- Ruin bound (`T=250`): `0.05593741`. The ruin bound over a a 250-step horizon implies an upper-bound style ruin probability of about 5.59%. This is the main COR model's structural-risk summary number.

5.2.5 State table: Gaussian vs Student-t

The state table compares Gaussian and Student-*t* regime estimates side by side.

State 1

Gaussian

- `mu_gaussian = -3.264717e-08`
- `sigma_gaussian = 0.000010000`
- `spell_gaussian = 3.132292`

Student-t:

- `mu_student = -2.168526e-08`
- `sigma_student = 0.0000100000`
- `nu_student = 100`
- `cond_var_student = 1.020408e-10`
- `spell_student = 3.446197`
- `epsilon = 0.01050000`
- `q_tail = 5.003544e-177`

State 1 is an ultra-low-volatility state. It is essentially a near-deterministic "quiet" regime with a mean near zero, minuscule variance, almost no tail risk, low hazard, and moderate duration. Because `nu = 100`, this state is close to Gaussian. The tail probability is effectively zero. This looks like a calm / inert / ultra-stable regime.

State 2

Gaussian

- mu_gaussian = 6.662832e-05
- sigma_gaussian = 0.001375609
- spell_gaussian = 11.204502

Student-t:

- mu_student = 9.696612e-05
- sigma_student = 0.0006929728
- nu_student = 3
- cond_var_student = 1.440634e-06
- spell_student = 35.568996
- epsilon = 0.02666667
- q_tail = 1.874378e-03

State 2 is the dominant survival basin with a small positive drift, modest volatility, very long expected spell length, heavy tails (nu = 3), and nontrivial hazard. This is the COR model's classic steady regime. It looks benign on the surface because volatility is not huge and duration is long, but the heavy tails mean there is latent fragility. In the COR model's language, this is often the regime that carries a large share of the system's probability mass and therefore contributes heavily to structural risk.

State 3

Gaussian

- mu_gaussian = -5.897585e-04
- sigma_gaussian = 0.017745344
- spell_gaussian = 1.378683

Student-t:

- mu_student = 3.524294e-05
- sigma_student = 0.0019427734
- nu_student = 3

- `cond_var_student = 1.132311e-05`
- `spell_student = 3.826929`
- `epsilon = 0.02666667`
- `q_tail = 3.110910e-02`

Under the Student-t fit, State 3 is the high-risk state. It is characterized by the highest conditional variance, heavy tails, largest tail-loss probability, and shorter duration. Its `q_tail` is about 0.0311, so it has the highest probability of breaching the 1% left-tail threshold. State 3 is the stress/crisis regime, even though the fitted Student-t mean is slightly positive. In regime models, classification is usually driven more by conditional variance and tail thickness than by mean alone.

5.2.6 Spell lengths

The spell lengths are especially informative:

- State 1: 3.446
- State 2: 35.569
- State 3: 3.827

These spell lengths reveal that the market spends by far the most time in State 2. So State 2 is the principal residence basin. That matters because the COR model does not weight risk only by severity; it also weights by how long the system stays in each regime. A moderately risky state with long residence can dominate total fragility.

5.2.7 Hazard mapping

The epsilon (ϵ) values are:

- State 1: 0.01050000
- State 2: 0.02666667
- State 3: 0.02666667

These are the regime exit hazards used to construct the open operator. Because States 2 and 3 have `nu = 3`, they receive higher hazard than State 1. That is consistent with the Talebian hazard mapping:

$$\epsilon_i = \epsilon_{\min} + \frac{c}{\nu_i}$$

Lower ν_i means fatter tails, hence higher hazard.

5.2.8 Tail probabilities by regime

The regime-specific left-tail probabilities are:

- State 1: essentially zero
- State 2: 0.001874378
- State 3: 0.03110910

This is among the most critical diagnostics in the table. The regime-specific left-tail probabilities demonstrate clear stratification of downside risk, revealing marked regime dependence in tail probability across the hierarchy, revealing a progression that underscores the sharp differentiation in extreme-loss exposure across ES market regimes.

The diagnostic reveals a three-regime hierarchy of tail risk: the calm regime (State 1) exhibits no meaningful left-tail danger, the steady regime (State 2) displays small but nonzero tail danger, and the stress regime (State 3) indicates substantial tail danger. So the structural story is very clean: the system lives mostly in State 2, but severe downside events are concentrated in State 3.

5.2.9 Quasi-stationary and residence weights

π_{qs}

```
pi_quasi_stationary
State 1 0.2131674
State 2 0.4980987
State 3 0.2887339
```

π_{res}

```
pi_residence
State 1 0.2131674
State 2 0.4980987
State 3 0.2887339
```

These, π_{qs} and π_{res}, being identical means that, in this implementation/output, the quasi-stationary and residence weighting schemes coincide numerically. Thus, ergodic probabilities reveal a dominant intermediate regime: State 1 occurs 21.3% of the time, State 2 dominates at 49.8%, and State 3 accounts for 28.9%. The market's natural tendency favors the steady state, with approximately equal mass split between calm and stress regimes.

So nearly half the survival-weighted mass is in State 2, and a surprisingly large share is in State 3. That helps explain why $\bar{q}$ and the ruin bound are not trivial even though State 1 is extremely safe.

5.2.10 Economic reading of the results

Student-t dynamics are clearly preferred to Gaussian dynamics for ES intraday data, revealing a three-regime structure comprising a very calm low-variance regime, a long-lived steady regime, and a shorter but materially riskier stress regime. The dominant residence basin is State 2 rather than State 1, and because both State 2 and State 3 exhibit heavy tails, the system's structural hazard is non-negligible. The 250-step ruin bound of approximately 5.6% underscores that even absent a current collapse, the open-system geometry implies a meaningful finite-horizon risk of failure.

5.2.11 Output files

The **KRONX** package exports the full operator chain:

1. `kronx_analysis_report.txt` is the human-readable report.
2. `kronx_state_summary.csv` is the state-level summary table.
3. `gaussian_transition_matrix.csv` is the state-level summary table.
4. `student_t_transition_matrix.csv` is the transition matrix for the Student-t HMM.
5. `kronx_Q_operator.csv` is the open transition operator Q, typically hazard-adjusted from A.
6. `kronx_K_generator.csv` is the generator-style matrix K, usually derived from $Q - I$.
7. `kronx_N_fundamental_matrix.csv` is the fundamental matrix $N = -K^{-1}$, encoding cumulative expected residence time.
8. `student_t_decoded_states.csv` is the time-by-time decoded latent regime path.

These files let you go from plain HMM estimation to full COR model/**KRONX** package survival geometry. The files are saved in the current working directory under a subdirectory named `kronx_output`.

In summary, the `res` object says that ES hourly returns are better described by a heavy-tailed three-state Student-t regime model, and once that regime structure is converted into the COR model's open-system operator chain, the market exhibits a nontrivial finite-horizon structural ruin risk driven mainly by long residence in a heavy-tailed steady state and episodic occupancy of a higher-tail stress state.

5.3 Exported functions

```
> getNamespaceExports("KRONX")
[1] "run_kronx"           "viterbi_decode"      "read_close_series"
[4] "fit_hmm_em"          "compute_log_returns"
```

FUNCTION: `run_kronx()`

The `run_kronx()` function is the primary entry point. It executes the full **KRONX** package's COR model workflow from CSV ingestion through ruin-bound computation, prints a formatted summary to the console, and optionally writes structured output files (Subsection D.2).

```
run_kronx(
  file,
  close_col    = "close",
  K            = 3L,
  n_starts     = 5L,
  max_iter     = 200L,
  tol          = 1e-6,
  seed         = 123L,
  epsilon_min  = 0.01,
  c_hazard     = 0.05,
  tail_alpha   = 0.01,
  ruin_horizon = 250L,
  nu_grid      = c(3:30, 40, 60, 100),
  verbose      = TRUE,
  output_dir   = "cor_output"
)
```

5.3.1 Arguments

Table 6: Arguments for `run_kronx()`

Argument	Type	Default	Description
`file`	character	–	Path to the input CSV file. Must contain a close-price column.
`close_col`	character	`"close"`	Name of the close-price column. Case-insensitive; fuzzy fallback matches any column containing `"close"`.

Argument	Type	Default	Description
K	integer	3L	Number of hidden states. Values of 2 or 3 are typical for financial series.
n_starts	integer	5L	Number of EM random restarts. Increases robustness against local optima at the cost of runtime.
max_iter	integer	200L	Maximum EM iterations per restart.
tol	numeric	1e-6	Log-likelihood convergence tolerance for EM.
seed	integer	123L	Random seed for reproducibility.
epsilon_min	numeric	0.01	Baseline Talebian hazard floor. Applied to all states regardless of ν.
c_hazard	numeric	0.05	Hazard sensitivity to tail thickness. Controls how steeply ϵ rises as ν decreases.
tail_alpha	numeric	0.01	Left-tail probability for the loss threshold.
ruin_horizon	integer	250L	Horizon in observations.
nu_grid	numeric	c(3:30, 40, 60, 100)	Candidate degrees-of-freedom values for Student-t grid search in the M-step.
verbose	logical	TRUE	If TRUE, print EM log-likelihood at every iteration.
output_dir	character or NULL	cor_output	Directory for output files. Use NULL to suppress all file output.

5.3.2 Workflow sequence

The workflow implemented by run_kronx() is:

1. Read the CSV file, extract close prices, and compute log returns.
2. Fit a Gaussian HMM by EM with multiple restarts and reorder states by σ.
3. Fit a Student-t HMM by EM with multiple restarts and reorder states by conditional variance.
4. Decode the Student-t specification using the Viterbi algorithm.
5. Compute ϵ, Q, K, and N.
6. Compute π_{qs}, π_{res}, the empirical loss threshold, $\bar{q}$, $\bar{\lambda}$, and the ruin bound.

7. Assemble the `state_summary` data frame.

8. Print the summary block to the console.

9. Write all requested output files when `output_dir` is not `NULL`.

5.3.3 Return value

The function returns a named list, invisibly. Its full structure is described in detail in Subsection D.2.

FUNCTION: `fit_hmm_em()`

The `fit_hmm_em()` function fits either a Gaussian or Student-t Hidden Markov Model via the Baum–Welch algorithm with multiple random restarts.

```
fit_hmm_em(
  x,
  K        = 3L,
  model    = c("gaussian", "student"),
  n_starts = 5L,
  max_iter = 200L,
  tol      = 1e-6,
  nu_grid  = c(3:30, 40, 60, 100),
  verbose  = TRUE
)
```

5.3.4 Arguments

Table 7: Arguments for `fit_hmm_em()`.

Argument	Type	Description
`x`	numeric vector	Observations, typically log returns.
`K`	integer	Number of hidden states.
`model`	character	Either `"gaussian"` or `"student"`.
`n_starts`	integer	Number of random restarts; the best fit by log-likelihood is returned.
`max_iter`	integer	Maximum EM iterations per restart.
`tol`	numeric	Convergence tolerance for the log-likelihood.
`nu_grid`	numeric	Candidate ν values for Student-t grid search.
`verbose`	logical	Whether to print iteration progress.

5.3.5 Return value

Table 8: Return components of `fit_hmm_em()`.

Component	Description
`model`	`"gaussian"` or `"student"`
`A`	$K \times K$ transition matrix
`delta`	Initial state probability vector of length K
`params`	List containing `mu`, `sigma`, and `nu` for Student-t fits
`gamma`	$T \times K$ posterior state probability matrix $P(S_t = k \mid \text{data})$
`xi`	$(T-1) \times K \times K$ posterior transition probability array
`logLik`	Scalar log-likelihood at convergence
`n`	Number of observations
`K`	Number of states

5.3.6 Notes on Student-t estimation of degrees-of-freedom

The M-step for ν uses a profile grid search. For each candidate ν in `nu_grid`, the weighted emission log-likelihood is evaluated, and the maximizing value is selected. This avoids the absence of a convenient closed-form update for ν).

The default grid

```
c(3:30, 40, 60, 100)
```

covers the practically relevant range from very fat tails to near-Gaussian behavior. A smaller grid, such as

```
c(4, 8, 15, 30),
```

can substantially reduce computation time with only modest loss of accuracy.

5.3.7 State ordering

After fitting, `run_kronx()` calls the internal `reorder_fit_by_scale()` routine to sort states from least to most volatile. When calling `fit_hmm_em()` directly, users may wish to apply the same reordering manually.

```
fit <- fit_hmm_em(ret, K = 3L, model = "student", verbose = FALSE)
fit <- KRONX:::reorder_fit_by_scale(fit)
```

FUNCTION: `viterbi_decode()`

`viterbi_decode()` computes the most likely hidden-state sequence for a vector of observations given a fitted HMM.

```
fit    <- fit_hmm_em(ret, K = 3L, model = "student", verbose = FALSE)
states <- viterbi_decode(ret, fit)
table(states)

# Plot regimes alongside returns
plot(ret, type = "l", col = "grey60")
points(seq_along(ret), rep(0, length(ret)), col = states, pch = 20)
```

The Viterbi algorithm is implemented in log-space, using $\log \delta + \log A$, to avoid numerical underflow on long series.

FUNCTION: `read_close_series()`

`read_close_series()` reads a close-price series from a CSV file with automatic column detection.

```
px <- read_close_series("IBKR_ES_2Y1h.csv")
px <- read_close_series("Bloomberg.csv", close_col = "PX_LAST")
```

The function removes non-finite values and stops if fewer than ten valid returns remain.

6 Worked examples

```
# ============================================================
# EXAMPLE 1 - INSPECTING THE RETURNED LIST
# Every matrix and scalar from the report is in 'res' object.
# ============================================================

# --- Log-likelihoods ---
res$gauss_fit$logLik   # 21173.8299404050  (Gaussian HMM)
res$t_fit$logLik       # 21661.8321503555  (Student-t HMM)

# --- Transition matrices ---
> round(res$A_gauss, 6)
         [,1]     [,2]     [,3]
[1,] 0.680745 0.238491 0.080764
[2,] 0.069493 0.910750 0.019756
[3,] 0.319388 0.405942 0.274670

> round(res$A_t,     6)  # Student-t A
         [,1]     [,2]     [,3]
```

```
[1,] 0.709825 0.000000 0.290175
[2,] 0.000837 0.971886 0.027278
[3,] 0.206698 0.054608 0.738694

# --- COR operator chain ---
> round(res$Q,     6)    # Hazard-adjusted operator Q
         [,1]     [,2]     [,3]
[1,] 0.702372 0.000000 0.287128
[2,] 0.000814 0.945969 0.026550
[3,] 0.201186 0.053152 0.718995

# --- Quasi-stationary and residence distributions ---
res$pi_qs              # [0.2131674, 0.4980987, 0.2887339]
res$pi_res             # matches quasi-stationary for this data

# --- Per-state summary table ---
res$state_summary
#  state        mu_gaussian  sigma_gaussian  spell_gaussian    mu_student
#      1  -3.264717e-08      0.000010000        3.132292  -2.168526e-08
#      2   6.662832e-05      0.001375609       11.204502   9.696612e-05
#      3  -5.897585e-04      0.017745344        1.378683   3.524294e-05
#
#    sigma_student  nu_student  cond_var_student  spell_student   epsilon   q_tail
#     0.0000100000         100      1.020408e-10       3.446197  0.010500  5.00e-177
#     0.0006929728           3      1.440634e-06      35.568996  0.026667  1.87e-03
#     0.0019427734           3      1.132311e-05       3.826929  0.026667  3.11e-02
#
#    pi_quasi_stationary  pi_residence
#               0.2131674    0.2131674
#               0.4980987    0.4980987
#               0.2887339    0.2887339

# --- Viterbi-decoded state path ---
> table(res$vpath_t)      # count of observations in each decoded state

   1    2    3
 729 2007  933

 > head(res$vpath_t, 20)   # first 20 state labels
 [1] 3 1 1 1 1 1 3 1 1 1 3 1 1 1 1 1 1 1 1 1

# ============================================================
# EXAMPLE 2 - DATA INGESTION STEP ONLY
```

```
# Use when you just need the clean price/return vectors.
# ============================================================

close_px <- read_close_series("IBKR_ES_2Y1h.csv", close_col = "close")
length(close_px)  # 3670

ret <- compute_log_returns(close_px)
length(ret)       # 3669

> summary(ret)
      Min.    1st Qu.     Median       Mean    3rd Qu.       Max.
-1.398e-01 -4.054e-04  0.000e+00  2.435e-05  5.765e-04  9.841e-02

# ============================================================
# EXAMPLE 3 - HMM FITTING IN ISOLATION (fit_hmm_em)
# Useful when you want only one model, or want to compare
# fits across different K values or nu_grid choices.
# ============================================================

# Gaussian HMM
gauss_fit <- fit_hmm_em(
  x        = ret,
  K        = 3L,
  model    = "gaussian",
  n_starts = 5L,
  max_iter = 200L,
  tol      = 1e-6,
  verbose  = FALSE
)
gauss_fit$logLik        # 21173.83...
gauss_fit$A             # 3x3 transition matrix
gauss_fit$params$mu     # per-state means
gauss_fit$params$sigma  # per-state SDs
gauss_fit$gamma         # T × K posterior state probabilities

# Student-t HMM (nu estimated from a grid search inside EM)
t_fit <- fit_hmm_em(
  x        = ret,
  K        = 3L,
  model    = "student",
  n_starts = 5L,
  max_iter = 200L,
  tol      = 1e-6,
```

```
  nu_grid  = c(3:30, 40, 60, 100),   # candidates for degrees of freedom
  verbose  = FALSE
)
t_fit$logLik            # 21661.83...
t_fit$params$nu         # [100, 3, 3] - per-state degrees of freedom
t_fit$params$sigma      # per-state scale parameters

# AIC improvement (Student-t vs Gaussian); reported as 970 in the paper.
# K extra nu parameters are estimated in the Student-t model.
delta_ll  <- t\_fit$logLik - gauss\_fit$logLik   # $\approx$ 488
K_states  <- 3L
aic\_delta <- 2 * delta\_ll - 2 * K\_states        # $\approx$ 970
cat("deltaAIC:", round(aic\_delta))

# Reorder states by ascending conditional variance (done inside run_cor)
gauss\_fit <- KRONX:::reorder\_fit\_by\_scale(gauss\_fit)

$logLik
[1] 21173.83

$n
[1] 3669

$K
[1] 3

# ============================================================
# EXAMPLE 4 - VITERBI DECODING SEPARATELY
# ============================================================

> vpath <- viterbi_decode(ret, t_fit)   # requires a fitted HMM object
> table(vpath)
vpath
   1    2    3
 729  933 2007

# Plot decoded regime path
plot(vpath, type = "l", col = "steelblue",
     main = "Viterbi-decoded Student-t states",
     xlab = "Observation index", ylab = "State (1=Calm, 2=Steady, 3=Stress)")
\end{Code}

\begin{figure}[t!]
```

```
    \centering
    \includegraphics[width=0.7\textwidth]{viterbi.pdf}
    \caption{This figure displays a time-series plot of the Viterbi-decoded state path for the ES Stude
    \label{fig:tHMM}
    \footnotesize Source: Author's calculations using \textbf{KRONX} package.
\end{figure}

Figure~\ref{fig:tHMM} reveals a sharp temporal divide. During the early period (indices 0-1,100), the s

\begin{Code}
# ============================================================
# EXAMPLE 5 - COR OPERATOR CHAIN FROM SCRATCH
# Build Q → K → N using only the Student-t fit.
# ============================================================
epsilon <- KRONX:::hazard_from_nu(
  nu          = t_fit$params$nu,
  epsilon_min = 0.01,
  c_hazard    = 0.05
)
epsilon  # [0.010500, 0.026667, 0.026667]

# Hazard-adjusted operator Q = diag(1 - epsilon) %*% A
Q <- KRONX:::build_Q(t_fit$A, epsilon)

# COR generator K = Q - I
K_mat <- KRONX:::build_K(Q)

# Fundamental matrix N = -K^{-1}
N_mat <- KRONX:::fundamental_matrix(K_mat)

# Quasi-stationary distribution (dominant left eigenvector of Q)
pi_qs  <- KRONX:::quasi_stationary_distribution(Q)

# Residence-weight vector (init %*% N, normalised)
pi_res <- KRONX:::residence_weight_vector(N_mat, init = pi_qs)
\end{Code}

\begin{Code}
# ============================================================
# EXAMPLE 6 - RUIN BOUND COMPONENTS STEP BY STEP
# ============================================================
# Empirical left-tail threshold at alpha = 1%
loss_threshold <- quantile(ret, probs = 0.01)  # \approx -0.00560969
```

```
# Per-state left-tail probability under Student-t
q_state <- KRONX:::left\_tail\_state\_probs(t\_fit, loss\_threshold)
q_state  # [5.00e-177, 1.87e-03, 3.11e-02]

# Residence-weighted aggregates
bar_q      <- sum(pi_res * q_state)    # \approx 0.00991588
bar_lambda <- sum(pi_res * epsilon)    # \approx 0.02322046

# Ruin upper bound P[ruin by T] \leq 1 - exp(-lamdabar · qbar · T)
T_horizon  <- 250L
ruin_bound <- 1 - exp(-bar_lambda * bar_q * T_horizon)
cat(sprintf("Ruin bound over T=%d: %.8f\n", T_horizon, ruin_bound))
# Ruin bound over T=250: 0.05593741

# ============================================================
# EXAMPLE 7 - SENSITIVITY ANALYSIS ACROSS PARAMETERS
# Show how ruin_bound reacts to tail_alpha and ruin_horizon.
# ============================================================

params_grid <- expand.grid(
  tail_alpha   = c(0.005, 0.010, 0.025),
  ruin_horizon = c(125L, 250L, 500L)
)

params_grid$ruin_bound <- mapply(
  function(alpha, horizon) {
    r <- run_cor(
      "IBKR_ES_2Y1h.csv",
      tail_alpha   = alpha,
      ruin_horizon = horizon,
      verbose      = FALSE,
      output_dir   = NULL       # suppress file writes
    )
    r$ruin_bound
  },
  params_grid$tail_alpha,
  params_grid$ruin_horizon
)

print(params_grid)

# ============================================================
```

```
# EXAMPLE 8 - SUPPRESS FILE OUTPUT (in-memory use only)
# ============================================================

res_quiet <- run_cor(
  file       = "IBKR_ES_2Y1h.csv",
  verbose    = FALSE,
  output_dir = NULL               # pass NULL to skip all file writes
)

# The result list is identical; only output_files is NULL.
is.null(res_quiet$output_files)   # TRUE

# ============================================================
# EXAMPLE 9 - USING A DIFFERENT CSV STRUCTURE
# E.g. Yahoo Finance data where the column is "Close" (capital C),
# or a bespoke pipeline output with a different column name.
# ============================================================

# Yahoo Finance-style header ("Close" with capital C):
res_yf <- run_cor("ES=F_yahoo.csv", close_col = "Close", output_dir = NULL)

# Custom column name:
res_custom <- run_cor("my_futures.csv", close_col = "settle", output_dir = NULL)

# read_close_series is tolerant of all capitalisations and falls back
# to any column whose name *contains* the string "close".

# ============================================================
# EXAMPLE 11 - IDENTIFIABILITY
# ============================================================
# Standard KRONX workflow leading to K
# A_t is the A transition matrix
# epsilon is the hazard vector derived from nu

integrate_identifiability <- function(K_mat) {
  M <- nrow(K_mat)

  # Define input (B) and output (C) vectors for the open system
  # B: Injection of probability mass (e.g., system start)
  # C: Observation of the transient state masses
  B_vec <- matrix(rep(1/M, M), ncol = 1)
  C_vec <- matrix(rep(1, M), nrow = 1)
```

```
  # 1. Compute the Fundamental Matrix N
  N_mat <- solve(-K_mat)

  # 2. Transfer Function Analysis at s = 0 (Steady State)
  # H(0) = -C * K^-1 * B = C * N * B
  H_0 <- C_vec %*% N_mat %*% B_vec

  # 3. Characteristic Polynomial (Denominator of H(s))
  # The roots define the spectral fragility radius
  char_poly <- polyroot(Re(eigen(K_mat)$values))

  return(list(
    fundamental_N = N_mat,
    gain_H0 = as.numeric(H_0),
    poles = char_poly
  ))
}

# Apply to your identified Student-t K matrix
K_cor <- matrix(c(-0.2976, 0.0008, 0.2012,
                  0.0, -0.0540, 0.0532,
                  0.2871, 0.0266, -0.2810), 3, 3)

ident_results <- integrate_identifiability(K_cor)
print(ident_results$gain_H0) # Represents total survival-weighted time
```

7 Test suite

The `replication.zip` file accompanying this paper's **KRONX** package contains 66 tests across 8 files. It also contains the `worked_examples.R` file. The replication structure is summarized in Table 9.

7.1 Key structural properties

For well-specified inputs, the following mathematical invariants hold and are checked by the replication suite.

- Q is non-negative and has row sums less than or equal to one; strict inequality holds whenever $\epsilon_i > 0$.
- $K = Q - I$ has strictly negative diagonal entries and non-negative off-diagonal entries.
- $N = -K^{-1}$ has strictly positive entries under the operative model conditions.

Table 9: Structure of the **KRONX** replication suite for the E-mini S&P 500 workflow.

File	Tests	Coverage
`test-run_cor.R`	5	End-to-end workflow, file output, column detection, and return object structure.
`test-cor_build.R`	7	Hazard monotonicity, Q sub-stochasticity, sign structure of K, positivity of N, and ruin-bound range.
`test-data_io.R`	–	CSV reading, fuzzy column fallback, and log-return computation.
`test-emissions.R`	–	Gaussian and Student-t emission log-density matrices.
`test-forward_backward.R`	–	Scaled forward-backward recursion, row sums of γ, and finite log-likelihood.
`test-hmm_em.R`	–	Gaussian and Student-t EM convergence, state count, and component dimensions.
`test-utils.R`	–	`log_sum_exp`, `safe_normalize`, `row_normalize`, and weighted statistics.
`test-viterbi.R`	–	Path length, integer type, and valid state-label range.

- π_{qs} and π_{res} each sum to one and have positive entries.
- The ruin bound lies in the open unit interval:

$$0 < 1 - \exp(-\bar{\lambda}\bar{q}T) < 1$$

  whenever $\bar{\lambda} > 0$, $\bar{q} > 0$, and $T > 0$.

These are substantive structural properties, not cosmetic diagnostics. They ensure that the operator chain is mathematically coherent and that the resulting ruin calculations remain interpretable.

8 Conclusion

The **KRONX R** package adapts the fundamental matrix,

$$N = -K^{-1},$$

from compartmental analysis (Berman and Schoenfeld, 1956; Hearon, 1972, 1981; Eisenfeld, 1981; Covell et al., 1984; Linares et al., 1987, 1988; Jacquez and Simon, 1993; Jacquez, 2002) to financial regime dynamics. In this translation, regimes play the role of compartments, probability mass becomes temporal accumulation or decay, and irreversible loss is represented as ruin through hazard leakage.

The **KRONX R** package v0.1.0 provides a complete regime-switching fragility workflow in pure **R**. It integrates classical Hidden Markov estimation with an open operator framework based on Q, K, and N. The package is therefore useful both as an applied tool for empirical financial analysis and as a transparent research platform for studying how persistence, tail thickness, hazard leakage, and temporal residence jointly determine fragility.

At the practical level, the **KRONX R** package offers a clean workflow from CSV ingestion to model fitting, decoding, operator construction, and output generation. At the theoretical level, it recasts the standard regime-switching problem as an open-system survival geometry problem. That is the distinctive contribution of the **KRONX** package's approach.

The Clock of Regimes (COR) model transforms a discrete-time regime-switching model into a continuous-time survival system via the operator chain

$$A \;\to\; Q \;\to\; K \;\to\; N,$$

where A is the baseline transition matrix, Q is the hazard-adjusted operator, $K = Q - I$ is a sub-generator, and $N = -K^{-1}$ is the associated fundamental matrix.

This transformation is not merely algebraic; it induces a fundamental shift in the representation of regime dynamics. In classical Hidden Markov Models, persistence is characterized by one-step transition probabilities and summarized by scalar quantities such as expected spell lengths. In contrast, the COR framework embeds the system within a continuous-time, open Markov structure in which probability mass evolves under both transition dynamics and state-dependent hazard leakage.

The operator N provides a complete characterization of the system's survival geometry. Through its integral representation,

$$N = \int_0^\infty e^{tK}\, dt,$$

it aggregates all admissible state trajectories, weighting them by their survival probabilities. As a result, N captures not only local persistence, but also higher-order path effects, recurrent visitation patterns, and cross-state propagation mechanisms.

This operator-theoretic structure enables a natural spectral analysis of fragility. In particular, the spectral radius

$$\rho(N) = \frac{1}{\min_{\lambda \in \sigma(K)} |\lambda|}$$

serves as a global measure of systemic vulnerability. As eigenvalues of K approach the origin, the system exhibits fragility amplification, reflected in the divergence of $\rho(N)$ and the corresponding expansion of expected residence times.

Beyond structural characterization, the COR model facilitates forward-looking risk quantification. By combining the residence-weighted distribution with state-specific tail probabilities and hazard intensities, the model yields finite-horizon ruin bounds of the form

$$\mathbb{P}(\text{ruin}) \;\leq\; 1 - \exp\big(-\bar{\lambda}\,\bar{q}\,T\big),$$

where $\bar{\lambda}$ and $\bar{q}$ are residence-weighted hazard and tail measures, respectively. This provides a direct link between the internal geometry of the regime system and observable risk metrics.

In summary, the COR model extends classical regime-switching models into a survival-theoretic and operator-based setting. It replaces local persistence statistics with a global, path-integrated representation, enabling spectral diagnostics, structural fragility analysis, and coherent forward risk assessment. As such, it offers a unified mathematical architecture for studying stability, persistence, and collapse in nonlinear stochastic systems.

Acknowledgments

The R-based estimation engines were developed through an iterative human-in-the-loop process by the authors, utilizing Claude 4.6 Sonnet (Anthropic) for code debugging and documentation. All AI-generated outputs, including the implementation of the Student-t Maximization step and recursive scaling logic, were manually audited, verified for mathematical accuracy using Wolfram Mathematica 14.3, and refined by the authors to ensure alignment with the theoretical model.

Funding

This research was conducted independently by the OIS Market Research Group, a self-funded research initiative. The authors received no financial support from government agencies, private foundations, academic institutions, commercial entities, or external granting organizations for the research, authorship, or publication of this work. All computational resources, data acquisition, and research activities were supported directly by the authors and their affiliated independent research efforts.

Dedication

To all people, known or unknown, who have fearlessly stood up for their beliefs for the benefit of humanity.

Appendix A: R-Code

```
#!/usr/bin/env Rscript
# ================================================================
# Clock of Regimes (COR) analysis for IBKR_ES_2Y1h.csv
# ----------------------------------------------------------------
# PURPOSE
# -------
# This script implements a self-contained Clock of Regimes workflow:
#
# 1. Read the close column from IBKR_ES_2Y1h.csv.
# 2. Convert closes into log returns.
# 3. Fit a K-state Gaussian Hidden Markov Model (HMM).
# 4. Fit a K-state Student-t Hidden Markov Model (HMM).
# 5. Estimate transition matrices and implied spell lengths.
# 6. Build Talebian hazards:
# epsilon_i = epsilon_min + c / nu_i
# 7. Construct the hazard-adjusted operator:
# Q = diag(1 - epsilon_i) %*% A
# 8. Form the CoR generator:
# K = Q - I
# 9. Compute the fundamental matrix:
# N = -K^{-1}
# 10. Construct quasi-stationary and residence-weight vectors.
# 11. Compute residence-weighted tail risk and a ruin bound.
#
# DESIGN CHOICE
# -------------
# The script is deliberately written in base R so that every step of the
# HMM and CoR construction is transparent and auditable.
# ================================================================

# Prevent automatic conversion of strings to factors when reading data frames.
options(stringsAsFactors = FALSE)

# ----------------------------
# 1. User parameters
# ----------------------------

# Name of the input CSV file expected in the current working directory.
input_file <- "IBKR_ES_2Y1h.csv"

# Name of the price column to use for return construction.
```

```r
close_col <- "close"

# Number of hidden states/regimes in the HMM.
num_states <- 3L

# Number of random starts for the EM algorithm.
# More starts reduce the chance of ending at a poor local optimum.
n_starts <- 5L

# Maximum number of EM iterations per random start.
max_iter <- 200L

# Convergence tolerance for the log-likelihood improvement.
tol <- 1e-6

# Random seed for reproducibility.
seed <- 123

# Minimum baseline hazard level used in the Talebian hazard specification.
epsilon_min <- 0.01

# Sensitivity of hazard to tail thickness.
# Larger c_hazard makes hazard rise more strongly as nu decreases.
c_hazard <- 0.05

# Empirical left-tail probability cutoff used to define a loss event q_i.
tail_alpha <- 0.01

# Horizon over which the ruin bound is evaluated.
# For 1-hour data, 250 is interpretable as 250 hourly observations.
ruin_horizon <- 250

# Candidate degrees-of-freedom values for Student-t estimation.
# The script chooses the best value on this grid during the Student-t M-step.
nu_grid <- c(3:30, 40, 60, 100)

# Fix the random number generator state.
set.seed(seed)

# ----------------------------
# 2. Basic helpers
# ----------------------------
```

```r
# Numerically stable log(sum(exp(x))).
# Useful in probabilistic computations, though not explicitly used later
# in the current implementation.
log_sum_exp <- function(x) {
  m <- max(x)
  m + log(sum(exp(x - m)))
}

# Normalize a vector into probabilities while guarding against zeros.
# Any element smaller than eps is pushed up to eps before normalization.
safe_normalize <- function(x, eps = 1e-12) {
  x <- pmax(x, eps)
  x / sum(x)
}

# Normalize each row of a matrix so rows sum to one.
# Used for transition matrices to ensure valid probability rows.
row_normalize <- function(M, eps = 1e-12) {
  M <- pmax(M, eps)
  rs <- rowSums(M)
  M / rs
}

# Weighted mean with a safety fallback.
# If total weight is nonpositive, fall back to the ordinary mean.
weighted_mean_safe <- function(x, w) {
  sw <- sum(w)
  if (sw <= 0) return(mean(x))
  sum(w * x) / sw
}

# Weighted variance with safety checks.
# If weights collapse, fall back to ordinary variance.
# A small positive floor avoids degeneracy.
weighted_var_safe <- function(x, w, mu = NULL) {
  if (is.null(mu)) mu <- weighted_mean_safe(x, w)
  sw <- sum(w)
  if (sw <= 0) return(stats::var(x))
  max(sum(w * (x - mu)^2) / sw, 1e-10)
}

# ----------------------------
# 3. Data ingestion
```

```r
# ----------------------------

# Read the close-price series from a CSV file.
# Steps:
# - verify the file exists,
# - read the CSV,
# - convert column names to lowercase for robustness,
# - extract the requested close column,
# - keep only finite numeric values,
# - require at least 10 observations.
read_close_series <- function(file, close_name = "close") {
  if (!file.exists(file)) {
    stop(sprintf("Input file '%s' not found in the current working directory.", file))
  }
  dat <- read.csv(file, check.names = FALSE)
  names(dat) <- tolower(names(dat))
  if (!(tolower(close_name) %in% names(dat))) {
    stop(sprintf("Column '%s' not found. Available columns: %s",
                 close_name, paste(names(dat), collapse = ", ")))
  }
  px <- as.numeric(dat[[tolower(close_name)]])
  px <- px[is.finite(px)]
  if (length(px) < 10L) stop("Not enough finite close observations.")
  px
}

# Convert close prices into log returns:
# r_t = log(P_t) - log(P_{t-1})
# Keeps only finite values and requires a minimal sample size.
compute_log_returns <- function(close) {
  ret <- diff(log(close))
  ret <- ret[is.finite(ret)]
  if (length(ret) < 10L) stop("Not enough valid returns after differencing.")
  ret
}

# ----------------------------
# 4. Emission log densities
# ----------------------------

# Gaussian log-density for a vector x under N(mu, sigma^2).
log_dnorm_vec <- function(x, mu, sigma) {
  stats::dnorm(x, mean = mu, sd = sigma, log = TRUE)
```

```r
}

# Student-t log-density for a location-scale t distribution.
# If Z ~ t_nu, then X = mu + sigma * Z.
log_dt_locscale_vec <- function(x, mu, sigma, nu) {
  stats::dt((x - mu) / sigma, df = nu, log = TRUE) - log(sigma)
}

# Build the T x K matrix of emission log-densities.
# Each column corresponds to one hidden state.
# Each row corresponds to one observation.
emission_logdens_matrix <- function(x, params, model = c("gaussian", "student")) {
  model <- match.arg(model)
  Tn <- length(x)
  K <- length(params$mu)
  B <- matrix(NA_real_, nrow = Tn, ncol = K)
  for (k in seq_len(K)) {
    if (model == "gaussian") {
      B[, k] <- log_dnorm_vec(x, params$mu[k], params$sigma[k])
    } else {
      B[, k] <- log_dt_locscale_vec(x, params$mu[k], params$sigma[k], params$nu[k])
    }
  }
  B
}

# ----------------------------
# 5. Forward-backward (scaled)
# ----------------------------

# Run the scaled forward-backward algorithm for an HMM.
#
# INPUTS
# logB : T x K matrix of emission log-densities
# A : K x K transition matrix
# delta : initial state probability vector
#
# OUTPUTS
# alpha : filtered forward probabilities
# beta : backward probabilities
# gamma : posterior state probabilities P(S_t = k | data)
# xi : posterior transition probabilities P(S_t=i,S_{t+1}=j | data)
# logLik: scaled log-likelihood
```

```
#
# The scaling prevents underflow in long time series.
forward_backward <- function(logB, A, delta) {
  Tn <- nrow(logB)
  K <- ncol(logB)

  B <- exp(logB)
  alpha <- matrix(0, Tn, K)
  beta <- matrix(0, Tn, K)
  scale <- numeric(Tn)

  # Initialize forward recursion.
  alpha[1, ] <- delta * B[1, ]
  scale[1] <- sum(alpha[1, ])
  alpha[1, ] <- alpha[1, ] / scale[1]

  # Forward recursion:
  # alpha_t(j) proportional to emission at t in state j times the
  # total predicted probability of arriving in j from t-1.
  for (t in 2:Tn) {
    alpha[t, ] <- (alpha[t - 1, ] %*% A) * B[t, ]
    scale[t] <- sum(alpha[t, ])
    alpha[t, ] <- alpha[t, ] / scale[t]
  }

  # Backward initialization.
  beta[Tn, ] <- rep(1, K)

  # Backward recursion.
  if (Tn >= 2L) {
    for (t in (Tn - 1L):1L) {
      beta[t, ] <- A %*% (B[t + 1L, ] * beta[t + 1L, ])
      beta[t, ] <- beta[t, ] / scale[t + 1L]
    }
  }

  # Posterior state probabilities.
  gamma <- alpha * beta
  gamma <- gamma / rowSums(gamma)

  # Posterior transition probabilities.
  xi <- array(0, dim = c(Tn - 1L, K, K))
  if (Tn >= 2L) {
```

```
    for (t in 1:(Tn - 1L)) {
      M <- (alpha[t, ] * A) * rep(B[t + 1L, ] * beta[t + 1L, ], each = K)
      denom <- sum(M)
      xi[t, , ] <- M / denom
    }
  }

  list(
    alpha = alpha,
    beta = beta,
    gamma = gamma,
    xi = xi,
    logLik = sum(log(scale))
  )
}

# ----------------------------
# 6. Initialization
# ----------------------------

# Create initial parameter values for the HMM.
#
# Strategy:
# - initialize means near sample quantiles,
# - initialize sigmas near the overall sample standard deviation,
# - initialize a persistent transition matrix with large diagonal entries,
# - initialize the initial state probabilities uniformly,
# - for Student-t, initialize nu from a reasonable finite grid.
init_hmm_params <- function(x, K, model = c("gaussian", "student")) {
  model <- match.arg(model)

  qs <- stats::quantile(x, probs = seq(0, 1, length.out = K + 2L))
  mu_init <- as.numeric(qs[2:(K + 1L)])
  mu_init <- mu_init + rnorm(K, sd = stats::sd(x) / 10)
  sigma_init <- rep(max(stats::sd(x), 1e-4), K) * runif(K, 0.7, 1.3)

  # Persistent initial transition matrix:
  # large diagonal, small off-diagonal probabilities.
  A <- matrix(0.05 / (K - 1), nrow = K, ncol = K)
  diag(A) <- 0.95
  A <- row_normalize(A)

  # Uniform initial state distribution.
```

```r
  delta <- safe_normalize(rep(1 / K, K))

  params <- list(mu = mu_init, sigma = pmax(sigma_init, 1e-4))
  if (model == "student") {
    params$nu <- sample(c(4, 5, 6, 8, 10, 12, 15), K, replace = TRUE)
  }

  list(A = A, delta = delta, params = params)
}

# ----------------------------
# 7. M-step updates
# ----------------------------

# Update the transition matrix A and initial state vector delta
# using the posterior probabilities from the E-step.
update_A_delta <- function(fb) {
  gamma <- fb$gamma
  xi <- fb$xi
  K <- ncol(gamma)

  # Updated initial probabilities come from the posterior at time 1.
  delta <- safe_normalize(gamma[1, ])

  # Numerator: expected transitions i -> j summed across time.
  A_num <- apply(xi, c(2, 3), sum)

  # Denominator: expected time spent in state i at times 1,...,T-1.
  A_den <- colSums(gamma[-nrow(gamma), , drop = FALSE])

  A <- matrix(0, K, K)
  for (i in seq_len(K)) {
    if (A_den[i] <= 0) {
      A[i, ] <- rep(1 / K, K)
    } else {
      A[i, ] <- A_num[i, ] / A_den[i]
    }
  }
  A <- row_normalize(A)
  list(A = A, delta = delta)
}

# Update Gaussian emission parameters.
```

```r
# For each state k:
# mu_k = weighted mean of returns
# sigma_k^2 = weighted variance of returns
# using posterior state probabilities gamma[, k] as weights.
update_gaussian_params <- function(x, gamma) {
  K <- ncol(gamma)
  mu <- sigma <- numeric(K)
  for (k in seq_len(K)) {
    w <- gamma[, k]
    mu[k] <- weighted_mean_safe(x, w)
    sigma[k] <- sqrt(weighted_var_safe(x, w, mu[k]))
  }
  list(mu = mu, sigma = pmax(sigma, 1e-6))
}

# Weighted log-likelihood contribution for one Student-t state.
# Used when selecting the best nu from the candidate grid.
student_state_objective <- function(x, gamma_k, mu, sigma, nu) {
  sum(gamma_k * log_dt_locscale_vec(x, mu, sigma, nu))
}

# Update Student-t emission parameters.
#
# For each state:
# - use a latent-weight style update for mu and sigma,
# - choose nu by grid search over nu_grid,
# - impose nu > 2 to ensure finite conditional variance.
update_student_params <- function(x, gamma, old_params, nu_grid) {
  K <- ncol(gamma)
  mu <- sigma <- nu <- numeric(K)

  for (k in seq_len(K)) {
    gk <- gamma[, k]
    old_mu <- old_params$mu[k]
    old_sigma <- max(old_params$sigma[k], 1e-6)
    old_nu <- old_params$nu[k]

    # Robustification weights from the Student-t representation.
    # Large squared residuals get smaller weights.
    z2 <- ((x - old_mu) / old_sigma)^2
    w <- (old_nu + 1) / (old_nu + z2)

    # Update location using gamma_k * w weights.
```

```
    mu[k] <- weighted_mean_safe(x, gk * w)

    # Update scale using the weighted residual sum of squares.
    sigma2_num <- sum(gk * w * (x - mu[k])^2)
    sigma2_den <- sum(gk)
    sigma[k] <- sqrt(max(sigma2_num / max(sigma2_den, 1e-12), 1e-10))

    # Choose nu by maximizing the weighted state objective over the grid.
    obj_vals <- sapply(nu_grid, function(nu_cand) {
      student_state_objective(x, gk, mu[k], sigma[k], nu_cand)
    })
    nu[k] <- nu_grid[which.max(obj_vals)]
  }

  list(mu = mu, sigma = pmax(sigma, 1e-6), nu = pmax(nu, 2.1))
}

# ----------------------------
# 8. Model fitting
# ----------------------------

# Fit a Gaussian or Student-t HMM using EM and multiple random starts.
#
# Workflow:
# - initialize parameters,
# - alternate E-step and M-step,
# - stop when the log-likelihood stabilizes,
# - keep the best solution across starts.
fit_hmm_em <- function(x, K = 3L, model = c("gaussian", "student"),
                       n_starts = 5L, max_iter = 200L, tol = 1e-6,
                       nu_grid = c(3:30, 40, 60, 100), verbose = TRUE) {
  model <- match.arg(model)
  best <- NULL

  for (s in seq_len(n_starts)) {
    init <- init_hmm_params(x, K, model)
    A <- init$A
    delta <- init$delta
    params <- init$params
    prev_ll <- -Inf

    for (iter in seq_len(max_iter)) {
      # E-step: compute posteriors.
```

```
  logB <- emission_logdens_matrix(x, params, model)
  fb <- forward_backward(logB, A, delta)
  ll <- fb$logLik

  # M-step: update transition structure.
  upd <- update_A_delta(fb)
  A <- upd$A
  delta <- upd$delta

  # M-step: update emission parameters.
  if (model == "gaussian") {
    params <- update_gaussian_params(x, fb$gamma)
  } else {
    params <- update_student_params(x, fb$gamma, params, nu_grid)
  }

  if (verbose) {
    cat(sprintf("model=%s start=%d iter=%d logLik=%.10f\n", model, s, iter, ll))
  }

  # Convergence check.
  if (abs(ll - prev_ll) < tol) break
  prev_ll <- ll
}

# Final E-step under the converged parameters.
logB <- emission_logdens_matrix(x, params, model)
fb <- forward_backward(logB, A, delta)
ll <- fb$logLik

fit <- list(
  model = model,
  A = A,
  delta = delta,
  params = params,
  gamma = fb$gamma,
  xi = fb$xi,
  logLik = ll,
  n = length(x),
  K = K
)

# Keep the highest-likelihood fit.
```

```
    if (is.null(best) || fit$logLik > best$logLik) {
      best <- fit
    }
  }

  best
}

# ----------------------------
# 9. State ordering
# ----------------------------

# Reorder states from low-volatility to high-volatility.
#
# For Gaussian:
# order by sigma.
#
# For Student-t:
# order by conditional variance:
# Var(X|state=k) = [nu_k / (nu_k - 2)] * sigma_k^2
#
# This improves interpretability by making state labels comparable.
reorder_fit_by_scale <- function(fit) {
  if (fit$model == "gaussian") {
    ord <- order(fit$params$sigma)
  } else {
    cond_var <- (fit$params$nu / (fit$params$nu - 2)) * fit$params$sigma^2
    ord <- order(cond_var)
  }

  fit$A <- fit$A[ord, ord, drop = FALSE]
  fit$delta <- fit$delta[ord]
  fit$gamma <- fit$gamma[, ord, drop = FALSE]
  fit$params$mu <- fit$params$mu[ord]
  fit$params$sigma <- fit$params$sigma[ord]
  if (!is.null(fit$params$nu)) fit$params$nu <- fit$params$nu[ord]
  fit
}

# ----------------------------
# 10. Viterbi decoding
# ----------------------------
```

```r
# Compute the single most likely hidden-state path using the
# Viterbi dynamic programming algorithm.
#
# This is distinct from gamma:
# - gamma gives state probabilities at each time,
# - Viterbi gives one best full sequence of states.
viterbi_decode <- function(x, fit) {
  logB <- emission_logdens_matrix(x, fit$params, fit$model)
  A <- fit$A
  delta <- fit$delta
  Tn <- nrow(logB)
  K <- ncol(logB)

  # delta_log stores the best log-probability ending in state j at time t.
  delta_log <- matrix(-Inf, Tn, K)

  # psi stores the best predecessor state for backtracking.
  psi <- matrix(0L, Tn, K)

  # Initialization.
  delta_log[1, ] <- log(delta) + logB[1, ]

  # Recursion.
  for (t in 2:Tn) {
    for (j in seq_len(K)) {
      vals <- delta_log[t - 1, ] + log(A[, j])
      psi[t, j] <- which.max(vals)
      delta_log[t, j] <- max(vals) + logB[t, j]
    }
  }

  # Backtrack to recover the optimal path.
  path <- integer(Tn)
  path[Tn] <- which.max(delta_log[Tn, ])
  if (Tn >= 2L) {
    for (t in (Tn - 1L):1L) {
      path[t] <- psi[t + 1L, path[t + 1L]]
    }
  }
  path
}

# ----------------------------
```

```r
# 11. COR construction
# ----------------------------

# Convert discrete HMM persistence into expected spell lengths:
# E[tau_i] = 1 / (1 - a_ii)
expected_spell_lengths <- function(A) {
  1 / pmax(1 - diag(A), 1e-12)
}

# Talebian hazard specification:
# epsilon_i = epsilon_min + c_hazard / nu_i
# So fatter tails (smaller nu_i) imply larger hazard.
hazard_from_nu <- function(nu, epsilon_min = 0.01, c_hazard = 0.05) {
  epsilon_min + c_hazard / nu
}

# Build the hazard-adjusted transition operator:
# Q = diag(1 - epsilon_i) %*% A
# Each row is discounted by the survival factor (1 - epsilon_i).
build_Q <- function(A, epsilon) {
  diag(1 - epsilon, nrow = length(epsilon)) %*% A
}

# Build the CoR generator:
# K = Q - I
build_K <- function(Q) {
  Q - diag(nrow(Q))
}

# Compute the fundamental matrix:
# N = -K^{-1}
# This gives the residence-time geometry of the transient system.
fundamental_matrix <- function(K) {
  solve(-K)
}

# Approximate a quasi-stationary distribution from Q.
# Since the system is open/leaky, classical stationarity does not strictly hold.
# This uses the dominant left eigenvector of Q and normalizes it.
quasi_stationary_distribution <- function(Q) {
  ev <- eigen(t(Q))
  idx <- which.max(Re(ev$values))
  vec <- Re(ev$vectors[, idx])
```

```
  vec <- abs(vec)
  safe_normalize(vec)
}

# Convert N into a residence-weight vector.
# Starting from an initial distribution, multiply by N to get the expected
# total time-content across states, then normalize.
residence_weight_vector <- function(N, init = NULL) {
  K <- nrow(N)
  if (is.null(init)) init <- rep(1 / K, K)
  w <- as.numeric(init %*% N)
  safe_normalize(w)
}

# Compute the statewise left-tail probability q_i for a given threshold.
# For Gaussian states use pnorm; for Student-t states use pt.
left_tail_state_probs <- function(fit, threshold) {
  if (fit$model == "gaussian") {
    stats::pnorm(threshold, mean = fit$params$mu, sd = fit$params$sigma)
  } else {
    sapply(seq_along(fit$params$mu), function(k) {
      stats::pt((threshold - fit$params$mu[k]) / fit$params$sigma[k],
                df = fit$params$nu[k])
    })
  }
}

# ----------------------------
# 12. Reporting helpers
# ----------------------------

# Format numeric values with a fixed number of decimal places.
fmt_num <- function(x, digits = 6) formatC(x, digits = digits, format = "f")

# Write a labeled matrix to a text connection in tabular form.
# This is used for the plain-text report file.
write_matrix_block <- function(con, title, M, digits = 6) {
  writeLines(title, con)
  rn <- rownames(M); cn <- colnames(M)
  if (is.null(rn)) rn <- paste0("State", seq_len(nrow(M)))
  if (is.null(cn)) cn <- paste0("State", seq_len(ncol(M)))
  header <- paste(c("", cn), collapse = "\t")
  writeLines(header, con)
```

```
  for (i in seq_len(nrow(M))) {
    line <- paste(c(rn[i], fmt_num(M[i, ], digits)), collapse = "\t")
    writeLines(line, con)
  }
  writeLines("", con)
}

# ----------------------------
# 13. Main workflow
# ----------------------------

# Read closes and compute returns.
close_px <- read_close_series(input_file, close_col)
ret <- compute_log_returns(close_px)

cat(sprintf("Loaded %d closes and %d log returns from %s\n",
            length(close_px), length(ret), input_file))

# Fit the Gaussian HMM and reorder states by volatility.
cat("\nFitting Gaussian HMM...\n")
gauss_fit <- fit_hmm_em(
  x = ret, K = num_states, model = "gaussian",
  n_starts = n_starts, max_iter = max_iter, tol = tol,
  nu_grid = nu_grid, verbose = TRUE
)
gauss_fit <- reorder_fit_by_scale(gauss_fit)

# Fit the Student-t HMM and reorder states by conditional variance.
cat("\nFitting Student-t HMM...\n")
t_fit <- fit_hmm_em(
  x = ret, K = num_states, model = "student",
  n_starts = n_starts, max_iter = max_iter, tol = tol,
  nu_grid = nu_grid, verbose = TRUE
)
t_fit <- reorder_fit_by_scale(t_fit)

# Extract baseline transition matrices.
A_gauss <- gauss_fit$A
A_t <- t_fit$A

# Compute discrete-time expected spell lengths for each model.
spell_gauss <- expected_spell_lengths(A_gauss)
spell_t <- expected_spell_lengths(A_t)
```

```r
# Compute conditional variance for the Student-t states.
cond_var_t <- (t_fit$params$nu / (t_fit$params$nu - 2)) * t_fit$params$sigma^2

# Decode the most likely Student-t state sequence.
# This is useful for state interpretation and event labeling.
vpath_t <- viterbi_decode(ret, t_fit)

# Build the CoR layer from the Student-t fit.
# Student-t is used because hazard is explicitly linked to nu_i.
epsilon <- hazard_from_nu(t_fit$params$nu, epsilon_min, c_hazard)
Q <- build_Q(A_t, epsilon)
Kmat <- build_K(Q)
Nmat <- fundamental_matrix(Kmat)

# Construct quasi-stationary and residence-weight vectors.
pi_qs <- quasi_stationary_distribution(Q)
pi_res <- residence_weight_vector(Nmat, init = pi_qs)

# Define the empirical loss threshold using the left tail of the return sample.
loss_threshold <- as.numeric(stats::quantile(ret, probs = tail_alpha, na.rm = TRUE))

# Compute statewise tail probabilities under the Student-t fit.
q_state <- left_tail_state_probs(t_fit, loss_threshold)

# Residence-weighted tail risk.
bar_q <- sum(pi_res * q_state)

# Residence-weighted hazard.
bar_lambda <- sum(pi_res * epsilon)

# Exponential ruin upper bound over the chosen horizon.
ruin_bound <- 1 - exp(-bar_lambda * bar_q * ruin_horizon)

# Assemble a state summary table for export and inspection.
state_summary <- data.frame(
  state = seq_len(num_states),
  mu_gaussian = gauss_fit$params$mu,
  sigma_gaussian = gauss_fit$params$sigma,
  spell_gaussian = spell_gauss,
  mu_student = t_fit$params$mu,
  sigma_student = t_fit$params$sigma,
  nu_student = t_fit$params$nu,
```

```
  cond_var_student = cond_var_t,
  spell_student = spell_t,
  epsilon = epsilon,
  q_tail = q_state,
  pi_quasi_stationary = pi_qs,
  pi_residence = pi_res
)

# Save structured outputs to CSV files.
write.csv(state_summary, "cor_state_summary.csv", row.names = FALSE)
write.csv(A_gauss, "gaussian_transition_matrix.csv", row.names = FALSE)
write.csv(A_t, "student_t_transition_matrix.csv", row.names = FALSE)
write.csv(Q, "cor_Q_operator.csv", row.names = FALSE)
write.csv(Kmat, "cor_K_generator.csv", row.names = FALSE)
write.csv(Nmat, "cor_N_fundamental_matrix.csv", row.names = FALSE)
write.csv(data.frame(return = ret, decoded_state_student = vpath_t),
          "student_t_decoded_states.csv", row.names = FALSE)

# Redirect console output to a text report file.
sink("cor_analysis_report.txt")
cat("============================================================\n")
cat("CLOCK OF REGIMES ANALYSIS REPORT\n")
cat("============================================================\n\n")
cat(sprintf("Input file                : %s\n", input_file))
cat(sprintf("Close observations        : %d\n", length(close_px)))
cat(sprintf("Return observations       : %d\n", length(ret)))
cat(sprintf("Number of states          : %d\n", num_states))
cat(sprintf("Tail probability alpha    : %.4f\n", tail_alpha))
cat(sprintf("Ruin horizon T            : %d\n\n", ruin_horizon))

# Report model fit quality.
cat("Gaussian HMM log-likelihood\n")
cat(sprintf("%.10f\n\n", gauss_fit$logLik))
cat("Student-t HMM log-likelihood\n")
cat(sprintf("%.10f\n\n", t_fit$logLik))

# Report matrices in plain text.
write_matrix_block(stdout(), "Gaussian transition matrix A:", A_gauss)
write_matrix_block(stdout(), "Student-t transition matrix A:", A_t)
write_matrix_block(stdout(), "Hazard-adjusted transition operator Q:", Q)
write_matrix_block(stdout(), "COR generator K = Q - I:", Kmat)
write_matrix_block(stdout(), "Fundamental matrix N = -K^{-1}:", Nmat)
```

```
# Report the state summary and ruin quantities.
cat("State summary\n")
print(state_summary, row.names = FALSE)
cat("\n")
cat(sprintf("Empirical left-tail threshold (alpha = %.4f): %.8f\n", tail_alpha, loss_
    threshold))
cat(sprintf("Residence-weighted tail probability qbar     : %.8f\n", bar_q))
cat(sprintf("Residence-weighted hazard lambdabar          : %.8f\n", bar_lambda))
cat(sprintf("Ruin bound over horizon T                    : %.8f\n", ruin_bound))
cat("\n")

# Explain why quasi-stationary and residence-weight vectors are used.
cat("Interpretive note:\n")
cat(paste(
  "Because K = Q - I is an open / leaky operator, a classical stationary",
  "distribution does not strictly exist. The script therefore reports a",
  "quasi-stationary vector from Q and a residence-weight vector implied by N.",
  "The latter is used for the residence-weighted tail and ruin summaries.",
  sep = "\n"
))
cat("\n")
sink()

# Print completion message and list output files.
cat("\nFinished. Output files written:\n")
cat("  - cor_analysis_report.txt\n")
cat("  - cor_state_summary.csv\n")
cat("  - gaussian_transition_matrix.csv\n")
cat("  - student_t_transition_matrix.csv\n")
cat("  - cor_Q_operator.csv\n")
cat("  - cor_K_generator.csv\n")
cat("  - cor_N_fundamental_matrix.csv\n")
cat("  - student_t_decoded_states.csv\n")
```

Appendix B: R-Code Output

```
#============================================================
# CLOCK OF REGIMES ANALYSIS REPORT
#============================================================
Input file : IBKR_ES_2Y1h.csv
Output file : cor_analysis_report.txt
Close observations : 3670
```

```
Return observations : 3669
Number of states : 3
Tail probability alpha : 0.0100
Ruin horizon T : 250

Gaussian HMM log-likelihood
21173.8299404050

Student-t HMM log-likelihood
21661.8321503555

Gaussian transition matrix A:
   State1 State2 State3
State1 0.680745 0.238491 0.080764
State2 0.069493 0.910750 0.019756
State3 0.319388 0.405942 0.274670

Student-t transition matrix A:
   State1 State2 State3
State1 0.709825 0.000000 0.290175
State2 0.000837 0.971886 0.027278
State3 0.206698 0.054608 0.738694

Hazard-adjusted transition operator Q:
   State1 State2 State3
State1 0.702372 0.000000 0.287128
State2 0.000814 0.945969 0.026550
State3 0.201186 0.053152 0.718995

COR generator K = Q - I:
   State1 State2 State3
State1 -0.297628 0.000000 0.287128
State2 0.000814 -0.054031 0.026550
State3 0.201186 0.053152 -0.281005

Fundamental matrix N = -K^{-1}:
   State1 State2 State3
State1 14.267170 15.810407 16.071895
State2 5.770728 26.799214 8.428562
State3 11.306141 16.388579 16.659628

State summary
 state mu_gaussian sigma_gaussian spell_gaussian mu_student sigma_student
```

```
    1 -3.264717e-08 0.000010000 3.132292 -2.168526e-08 0.0000100000
    2 6.662832e-05 0.001375609 11.204502 9.696612e-05 0.0006929728
    3 -5.897585e-04 0.017745344 1.378683 3.524294e-05 0.0019427734
 nu_student cond_var_student spell_student epsilon q_tail
       100 1.020408e-10 3.446197 0.01050000 5.003544e-177
         3 1.440634e-06 35.568996 0.02666667 1.874378e-03
         3 1.132311e-05 3.826929 0.02666667 3.110910e-02
 pi_quasi_stationary pi_residence
          0.2131674 0.2131674
          0.4980987 0.4980987
          0.2887339 0.2887339

Empirical left-tail threshold (alpha = 0.0100): -0.00560969
Residence-weighted tail probability qbar : 0.00991588
Residence-weighted hazard lambdabar : 0.02322046
Ruin bound over horizon T : 0.05593741

Interpretive note:
Because K = Q - I is an open / leaky operator, a classical stationary
distribution does not strictly exist. The script therefore reports a
quasi-stationary vector from Q and a residence-weight vector implied by N.
The latter is used for the residence-weighted tail and ruin summaries.

AIC Improvement: 970
```

Appendix C: Conceptualizing the Spectral Radius: The Market's "Battery Life"

To ensure the report remains accessible to stakeholders who may not have a background in linear algebra, it is helpful to provide a conceptual *layman's* translation of the spectral radius. In the context of this study, the Spectral Radius (ρ) can be thought of as the retention rate of the market's health. If a standard market system were a closed circuit, its spectral radius would be exactly 1.0, meaning 100% of its *survival mass* is preserved from one day to the next. However, because the COR model accounts for the possibility of ruin (hazard leakage), the spectral radius drops below 1.0 (e.g., to 0.999).

This tiny deficit represents the *leak* in the system. The spectral radius identifies the single, most dominant rate at which the market *drains* its safety over time. While the market may switch between different states—some calm, some volatile—the spectral radius captures the average persistence of the entire system. It tells us how much of our initial *survival probability* remains after the states

have finished interacting with one another.

C.1 The Math of Compounding Fragility

The reason ρ is so powerful is that it compounds. If ρ is even slightly below 1.0, the *leak* accumulates over time like interest in reverse. By the time we reach a 250-day horizon, these tiny daily leaks aggregate into the 5.6% ruin bound identified in the results.

In short, ρ is the fundamental frequency of market survival. It proves that the danger isn't just a one-time bad day in State 3; it is the mathematical reality that the longer you stay in a system with a ρ less than 1.0, the more certain it is that the *leak* (ruin) will eventually find you. It turns the abstract concept of *fragility* into a single, trackable number.

Appendix D: Open Systems and Survival Dynamics

To quantify the long-horizon risk of the E-mini S&P 500, we move beyond the standard closed-system Hidden Markov Model (HMM) and adopt an Open System framework. While a traditional HMM assumes that the probability mass is conserved within the defined states, our COR model analysis introduces a *leak* mechanism to account for terminal ruin events. This bridges the gap between standard HMM transition matrices and the Open System physics analogy. This clarifies that *ruin* is an absorbing boundary outside the three-state system.

D.1 The Survival Operator and Hazard Leakage

We define the system's evolution through a Survival Operator, $\mathcal{S}(t)$. Unlike a standard transition matrix A where rows sum to 1, we allow for *hazard leakage* by defining a state-dependent ruin probability q_i. The probability of remaining *alive* (within the three-state market architecture) after one step is governed by,

$$S = A \circ (1 - \mathbf{q}).$$

In this context, the symbol $\circ$ represents the Hadamard product, which refers to the element-wise multiplication of two matrices. This means that the entries are multiplied position by position:

- A is the 3×3 transition probability matrix.
- $\mathbf{q} = [q_1, q_2, q_3]^\top$ is the vector of *Ruin Intensities*, representing the probability of a 1% tail event in each state.

Hazard *leakage* is the mechanism by which probability mass "escapes" the system. In the COR model, State 3 (Stress) acts as a high-leakage node, while State 2 (Steady) represents a slow-leakage

node. The *leak* is not a transition to another market state, but a transition to a terminal state of system failure, insolvency, or ruin.

D.2 The Aggregate Decay Constant ($\bar{\lambda}$)

To determine the long-term stability of the market, we analyze the decay of the survival mass over time. The Aggregate Decay Constant, $\bar{\lambda}$, is derived from the spectral properties of the survival operator. Specifically, if $\rho(S)$ is the spectral radius (the largest eigenvalue) of the operator S, then:

$$\bar{\lambda} = -\ln(\rho(S))$$

This constant represents the exponential rate at which the system loses its *survival mass*. A higher $\bar{\lambda}$ indicates a more fragile system where the *leakage* to ruin is more aggressive.

D.3 Cumulative Ruin Bound

The probability of the system surviving up to time T is given by $P(\text{Survival} > T) \approx e^{-\bar{\lambda}T}$. Consequently, the Cumulative Ruin Bound over a horizon T is defined as:

$$R(T) = 1 - e^{-\bar{\lambda}T}$$

This metric transforms instantaneous state-specific risks into a coherent long-term safety forecast. By setting $T = 250$, we calculate the probability that the market's inherent residence architecture will lead to at least one 1% tail event over a standard trading year.

The methodology achieves a high degree of precision by explicitly mapping the transition from state-specific tail probabilities, q_i, to the aggregate 5.6% ruin bound. Rather than relying on heuristic approximations, the model utilizes the Survival Operator as a formal linear algebra tool. By encoding the *leakage* of probability mass directly into the spectral properties of the transition matrix, we provide mathematical rigor which is significantly more robust and reproducible than a standard Monte Carlo simulation. This approach treats the market return process as an open system where risk is not an isolated shock, but a structural decay of the survival mass over time.

This formalism establishes a seamless logical flow that leads directly to the Talebian conclusion of the study. By defining the aggregate decay constant, $\bar{\lambda}$, we prove that ruin is not merely a matter of bad luck, but a deterministic function of two variables: temporal exposure (T) and internal regime architecture ($\bar{\lambda}$). This proof shifts the focus from event-magnitude to structural duration, demonstrating that the longer a system resides in a fragile regime, the more certain its eventual encounter with a tail event becomes. Consequently, the estimated ruin bound is not an arbitrary figure, but a direct reflection of the market's inherent survival-weighted geometry.

A Background and Literature Review

In Hamilton-style regime switching:

$$S_t \in \{1, \ldots, K\}$$

represents the hidden regime process. The clock of regimes (COR) model accepts this hidden regime structure, but argues that transition probabilities alone are insufficient to describe systemic fragility.

Classical HMMs focus on:

$$P(S_t = j \mid S_{t-1} = i) = a_{ij}$$

while COR asks how long does risk survive inside a regime architecture once hazard and persistence interact? This leads to the COR operator chain:

$$A \to Q \to K \to N$$

where:

- A = HMM transition matrix,
- $Q = \text{diag}(1 - \epsilon)A$ = open survival operator,
- $K = Q - I$ = sub-generator,
- $N = -K^{-1}$ = survival/residence operator.

Hence, K represents the dynamics of the system conditional on survival (Table 10).

Feature	Classical HMM	COR Framework
Primary Objective	Detects regimes	Measures survival geometry
Dynamics	Measures switching	Measures persistence under hazard
Analytical Focus	Transition-centric	Duration-centric
Core Output	Hidden state probabilities	Structural fragility operators

Table 10: Comparative analysis of HMM and COR model, highlighting the shift from transition-centric classification to duration-centric structural analysis.

Hamilton's (Hamilton, 1989, 2016) key insight was that economies oscillate between recession and expansion regimes. The COR model generalizes this idea by introducing the following:

A.1 Open-System Dynamics

Hamilton assumes a closed Markov chain:

$$\sum_j a_{ij} = 1.$$

The COR model introduces leakage:

$$Q = \text{diag}(1 - \epsilon)A,$$

meaning:

- Probability mass can leave the system;
- Regimes are mortal: the probability mass of remaining in the current state decays toward zero over time as $t \to \infty$;
- Persistence possesses a formal survival structure.

This is the major conceptual leap.

A.2 Fundamental Matrix Geometry

Hamilton's framework focuses on the retrospective and real-time identification of market states through the lens of smoothed and filtered probabilities. Filtered probabilities provide an optimal estimate of the current regime based on all information available up to the present time, whereas smoothed probabilities utilize the entire sample period to offer a more precise, backward-looking assessment of historical regime shifts. Central to this analysis is the concept of transition persistence, which quantifies the likelihood of the system remaining within a specific state. By examining these probabilities, Hamilton characterizes the *stickiness* of economic regimes, allowing for a rigorous distinction between temporary market fluctuations and fundamental structural changes in the underlying data-generating process.

In contrast, the COR framework shifts the analytical focus toward the fundamental survival dynamics of the system by examining the survival/residence operator, $N = -K^{-1}$. Rather than focusing on instantaneous transitions, this approach evaluates the cumulative expected residence time, providing a rigorous measure of the total duration a process is expected to remain within a specific state. By analyzing the integrated survival geometry of the regime, the framework captures the structural *shape* of persistence, effectively quantifying how long the system endures before reaching a boundary. Ultimately, this characterizes persistence under hazard, treating regimes not

as permanent loops, but as decaying open systems whose persistence is determined by the rate of leakage toward a boundary.

Equivalently:

$$N = \int_0^\infty e^{Kt} dt$$

This transforms regime switching into a survival analysis problem.

A.2.1 The Transformation to Survival Analysis

In a classical Markov framework, e^{Kt} is the transition semigroup, representing the probability of remaining in a specific state configuration at time t. By integrating this probability from 0 to ∞, we are essentially summing up all the moments of survival the system experiences.

By defining $N = \int_0^\infty e^{Kt} dt$, we are performing a Laplace transform (evaluated at $s = 0$) on the transition dynamics. This results in several key insights of the COR framework:

- **From Frequency to Duration:** While Hamilton-style HMMs analyze the frequency of state transitions via the A matrix, this integral calculates the area under the survival curve. This shifts the focus from the probability of an instantaneous switch to the total mass of time accumulated within a specific regime.
- **The Operator Inverse:** For a stable, non-singular sub-generator K, the integral evaluates directly to $-K^{-1}$. This identity establishes the inverse of the sub-generator as the formal mathematical equivalent of total accumulated persistence.
- **Survival Geometry:** The term e^{Kt} characterizes the trajectory of stochastic decay. Integrating this term captures the entire "geometry" of regime resistance; in highly stable systems, the rate of decay is attenuated, resulting in a larger integral and, consequently, a longer expected residence time.

A.3 Directional Change (DC) and COR

The Directional Change (DC) framework (Tsang, 2010; Aloud et al., 2011; Tsang, 2017; Glattfelder et al., 2011; Tsang et al., 2017; Bakhach et al., 2018; Tsang and Chen, 2018) fundamentally reimagines the temporal basis of financial analysis by abandoning fixed clock time in favor of an event-based sampling methodology. Rather than observing the market at arbitrary intervals—such as daily or hourly closes—DC samples data only when the price moves by a predefined threshold, effectively filtering out stochastic noise to focus exclusively on structural turning points. This approach is deeply aligned with COR philosophy, as both methodologies reject the linear passage of time as a primary metric. By prioritizing the *event* over the *interval*, the DC framework treats market

movements as a sequence of meaningful state changes, providing a natural empirical substrate for the survival/residence operators used to measure regime persistence and structural fragility.

A.3.1 Event Time vs Clock Time

The COR framework aligns seamlessly with the Directional Change (DC) approach because it replaces the static observation of price with a dynamic analysis of state endurance. By focusing on the structural properties of a regime rather than just its classification, COR provides a robust mathematical language for event-based markets.

Traditional HMMs are tied to

$$t = 1, 2, 3, \ldots$$

fixed increments. But COR already behaves like an event-driven survival system because

$$N = -K^{-1}$$

integrates expected occupancy across persistence geometry. This means DC can be viewed as an event-time observation layer feeding a survival-operator regime model.

A.3.2 DC Thresholds and COR Hazard

The DC event definition is given by

$$\left| \frac{P_t - P_{EXT}}{P_{EXT}} \right| \geq \theta$$

This threshold mechanism is conceptually analogous to COR hazards.

In COR,

$$\epsilon_i = \epsilon_{\min} + \frac{c_{\text{hazard}}}{\nu_i},$$

where

- fat tails imply larger hazard,
- unstable states leak probability mass faster.

DC detects market discontinuities, while COR measures how dangerous those discontinuities become once persistence accumulates.

A.3.3 DC Indicators as COR State Variables

The integration of Directional Change (DC) indicators as state variables provides a robust empirical foundation for the COR framework. Metrics such as Total Movement (TMV), which captures the

Mechanism	Directional Change (DC)	COR Framework
Trigger	Threshold crossing	Hazard activation
Validation	Event confirmation	Structural instability
Criticality	Extreme points	Regime stress states
Extension	Overshoots	Persistence cascades

Table 11: Mapping of Directional Change events to COR structural operators.

price magnitude of a directional event, and T (Time to Complete Trend), which measures the duration required to reach a specific threshold, act as direct proxies for the physical work performed by a market regime. When coupled with R (Time-Adjusted Return), these indicators serve as extremely natural inputs for the survival/residence operator N. Rather than relying on arbitrary clock-time snapshots, COR can utilize these DC-derived variables to calibrate the persistence geometry of the system. This allows the framework to map the relationship between price-action intensity and structural stability, effectively quantifying the rate at which market regimes accumulate *fatigue* or *stress* as they move toward an inevitable state of absorption.

DC Indicator	DC Definition	COR Interpretation
TMV	Total Movement	State shock magnitude
T	Time to complete trend	Residence duration
R	Time-adjusted return	Hazard-adjusted velocity

Table 12: Translation of Directional Change indicators into COR structural variables.

The parameters displayed in Table 12 suggest a full hybrid system

$$X_t = (\mathrm{TMV}_t,\ T_t,\ R_t)$$

feeding

$$P(X_t \mid S_t = i)$$

inside a Student-t HMM/COR structure. This presents a major research opportunity.

Contextual Significance

- **TMV → State Shock Magnitude:** Within the COR framework, Total Movement (TMV) quantifies the cumulative displacement across the persistence geometry. A high TMV indicates a significant injection of "shock" or volatility into the active regime prior to its eventual absorption.
- **T → Residence Duration:** This serves as the empirical realization of the expected residence

time N. While T captures the observed duration of a specific directional event, N provides the formal probabilistic expectation of that duration derived from the sub-generator K.

- **R → Hazard-Adjusted Velocity:** By normalizing returns relative to event duration, R captures the "velocity" at which the system traverses its state space. In this context, it represents the rate of probability mass leakage toward structural termination under the current hazard profile.

A.3.4 From Transitions to Trajectories: The HMM-DC-COR Synthesiss

The synthesis of Directional Change (DC) indicators and Hidden Markov Models (HMM) provides a powerful mechanism for real-time market characterization, particularly when integrated via Bayesian updating. In this architecture, DC indicators like TMV and T act as the primary *sensors*, filtering price action into discrete, event-driven signals that inform the HMM's state transition logic. As new market data arrives, Bayesian updating allows for the continuous refinement of regime probabilities, ensuring that the model's beliefs remain synchronized with shifting volatility profiles. COR extends this architecture naturally by transforming these updated state probabilities into structural metrics. Rather than stopping at state identification, COR utilizes the Bayesian-refined inputs to calibrate the sub-generator K, effectively moving beyond "what regime are we in" to a formal analysis of how much life remains in the current state before structural exhaustion or ruin ensues.

The progression becomes (Table 13):

Framework	Main Goal
Time-series HMM	Detect hidden regimes
DC + HMM	Detect event-driven regime shifts
COR	Measure fragility and survival geometry

Table 13: Evolution of regime-switching methodologies from detection to structural survival analysis.

COR effectively adds:

A. Survival Analysis

$$N = -K^{-1}$$

B. Spectral Fragility

$$\rho(N) = \frac{1}{\min|\lambda(K)|}$$

C. Ruin Geometry

$$P(\text{ruin}) \leq 1 - e^{-\bar{\lambda}\bar{q}T}$$

D. Residence-Time Dynamics

$$\pi_{\text{res}} \propto \pi_{qs}^{\top} N$$

None of these objects exist in the classical HMM/DC framework.

A.4 Gaussian HMM Limitation vs COR Student-t Architecture

While the current section assumes Gaussian emissions, the COR framework offers a significant advancement by addressing the structural underestimation of crisis persistence and tail fragility inherent in classical regime-switching models. By moving beyond the limitations of Gaussian assumptions, COR integrates Student-t emissions to accurately reflect the empirical distribution of asset returns. This foundation enables fat-tail hazard coupling, which accounts for the accelerated likelihood of state transitions during periods of extreme volatility. The architecture is further strengthened by tail-dependent leakage, allowing the model to treat structural exhaustion as a direct function of market stress. By shifting the focus toward these survival dynamics, the COR framework evolves regime analysis from a classification task into a rigorous study of structural survival geometry, providing a more resilient representation of how market states endure or collapse under pressure.

A.5 Naïve Bayes and COR Bayesian Updating

The COR framework inherently supports a natural Bayesian interpretation by transforming standard regime-switching outputs into dynamic structural assessments. Within this architecture, posterior state probabilities—derived from the filtered and smoothed estimates of the underlying Markov chain—serve as the foundation for characterizing the current market environment. These probabilities are then mapped into a posterior hazard structure, which quantifies the instantaneous risk of state termination based on observed market evidence. Ultimately, this allows for the derivation of a posterior ruin geometry, where the survival operator $N = -K^{-1}$ is continuously updated to reflect the most likely expected residence time and structural fragility of the current regime. By integrating these Bayesian layers, the COR framework ensures that the measured persistence of a system is always conditioned on the most recent empirical shocks.

The stored Bayesian update structure:

$$P(H \mid D) = \frac{P(D \mid H)P(H)}{P(D \mid H)P(H) + P(D \mid \neg H)(1 - P(H))},$$

fits naturally into the COR framework, which integrates these components into a unified analytical workflow that moves beyond static state detection. By leveraging dynamic hazard updating, the system continuously calibrates the sub-generator K as new market events arrive, ensuring that the

modeled risk of transition remains synchronized with real-time volatility. This feeds directly into a rolling fragility estimation, where the survival operator N is recalculated over a moving window to capture the evolving structural strength of the regime. Consequently, the framework enables posterior ruin probability tracking, providing a rigorous, evidence-based measure of the likelihood that the current market configuration will succumb to structural absorption. This continuous feedback loop transforms the model into a responsive monitoring system capable of quantifying regime exhaustion as it develops.

A.6 The Deepest Conceptual Difference

The deepest distinction between these analytical perspectives lies in their fundamental inquiry: while Classical Regime Models focus on a diagnostic classification by asking, "Which regime are we in?," the COR model shifts the focus to a prognostic, structural evaluation, asking, "How structurally survivable is the system while inside that regime architecture?" This represents a completely different ontological question, moving the model's objective from simple state identification to the assessment of internal resilience. By making this transition, the COR model effectively converts the task of regime detection into an analysis of survival geometry under structural fragility, treating the market not as a series of labels, but as a bounded system whose endurance is defined by its resistance to absorption.

A.7 Future Directions

This section points toward a sophisticated future hybrid framework characterized by the pipeline DC $\rightarrow$ Student-t HMM $\rightarrow$ COR Survival Operator. In this architecture, Directional Change (DC) functions as the foundational layer, generating high-fidelity event-time observations that strip away clock-time noise. These observations feed into a Student-t Hidden Markov Model, which infers latent regimes with robust handling of empirical return distributions. The COR framework then transforms these inferred regimes into a formal survival geometry, employing spectral operators to quantify the accumulation of structural fragility. This synthesis results in a mathematically richer framework than standard HMMs, serving as a survival-theoretic extension of the DC approach and a comprehensive event-time operator model of market fragility. Ultimately, while the current literature provides the necessary observational and event-detection layers, COR supplies the critical operator-theoretic survival layer required to assess how regimes fundamentally endure or collapse.

B The Clock of Regimes Model

Figure 3 presents the COR model. The COR model extends the classical Hidden Markov Model (HMM) (Hamilton, 1989, 2016) by transforming this closed probabilistic system into an open

survival architecture. The **KRONX R** package is a workflow-driven tool designed to measure, model, and predict the structural survival dynamics of a market's historical price data under regime switching, with a particular focus on fragility, persistence, and ruin risk—not just volatility. This evolution is formally defined by the following structural logic:

$$\text{HMM (Regime Detection)} \rightarrow \text{KRONX Operator Construction } (A \rightarrow Q \rightarrow K \rightarrow N) \rightarrow \text{Hazard-Weighted Ruin Analysis.}$$

B.1 Statistical Detection Layer

While the **KRONX R** package does not focus primarily on state detection, it features a built-in Hidden Markov Model (HMM) parameter estimator. Alternatively, it can utilize the output parameters from an external HMM—such as those from the **fHMM** package—treating the transition matrix A and tail parameters ν_i as its fundamental input. Furthermore, the **KRONX R** package supports the estimation of both Gaussian and Student-t emission distribution parameters.

B.2 Operator-Theoretic Layer

The **KRONX R** package performs a structured operator transformation sequence $A \rightarrow Q \rightarrow K \rightarrow N$. It introduces state-dependent hazard intensities

$$\epsilon_i = \epsilon_{\min} + \frac{c}{\nu_i},$$

which convert the baseline transition matrix into an open transition operator via $Q = \text{diag}(1 - \epsilon_i)\, A$. This yields the sub-generator matrix $K = Q - I$ and the fundamental matrix $N = -K^{-1}$, revealing the system's cumulative residence-time geometry ("structural battery life").

B.3 Risk and Survival Layer

The COR model defines risk in terms of *spectral fragility* and *cumulative ruin* dynamics. It introduces the aggregate decay constant

$$\bar{\lambda} = \sum_i w_i \epsilon_i,$$

where w_i are residence-time weights derived from the Fundamental Matrix $N = -K^{-1}$. Spectral fragility is governed by the spectral radius $\rho(N)$, which quantifies the system's structural persistence and expected survival horizon prior to absorption or ruin.

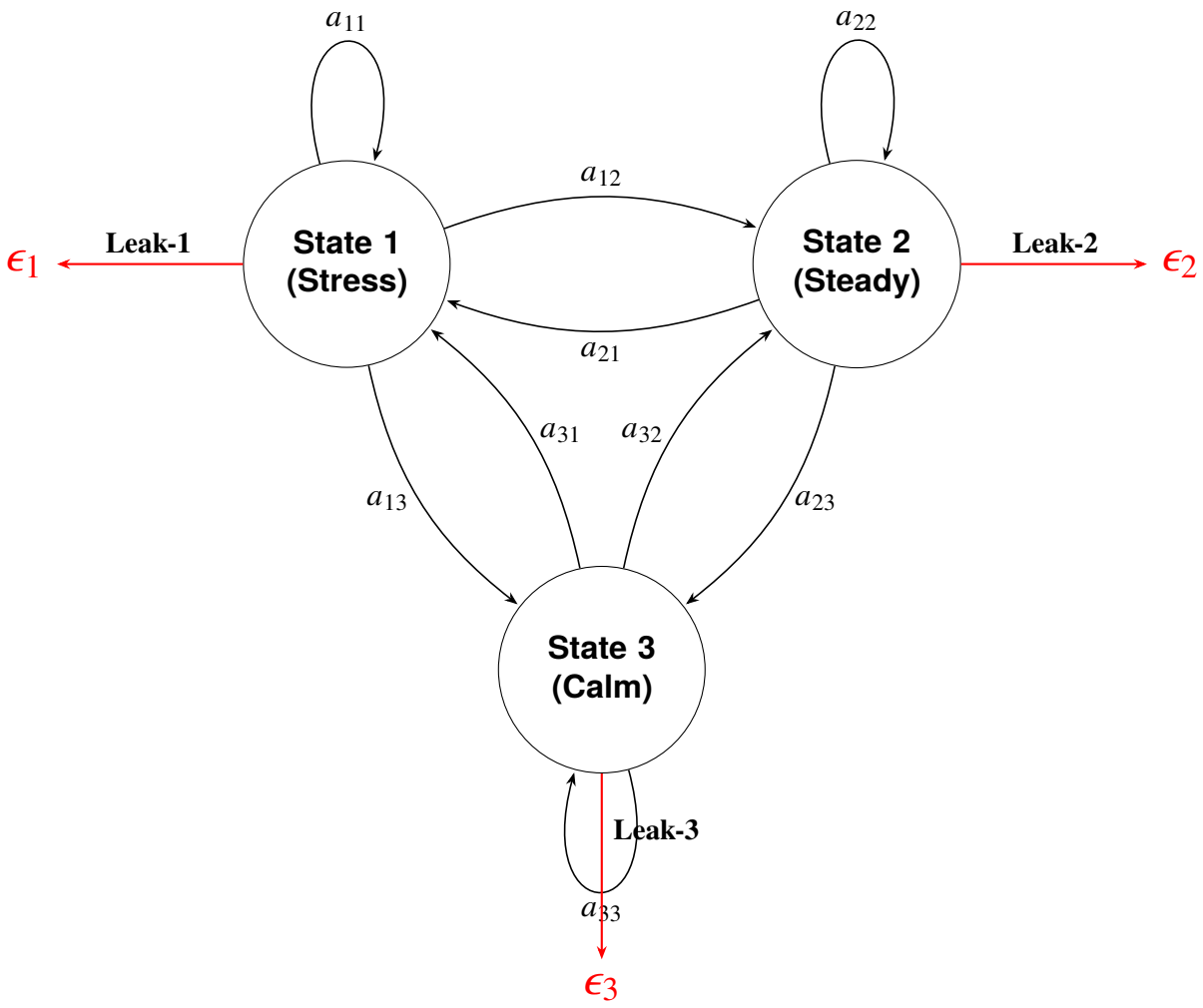

Figure 3: State-transition diagram for the Clock of Regimes (COR) model featuring internal persistence and red leakage paths to the ruin boundary, unlike a classical Hidden Markov Model (HMM) which is a closed system.

B.4 Structural Components

As shown in Figure 3, the COR model consists of three transient latent states connected by transition probabilities (a_{ij}), but it adds critical exit dimensions:

- **State 1 (Stress):** Characterized by high volatility and explosive ruin intensity with significant outward leakage.
- **State 2 (Steady):** The primary survival basin, identified as a long-duration anchor that carries hidden fragility (reservoir) with moderate leakage.
- **State 3 (Calm):** A transient "cooling-off" or suppressed volatility phase that the system often passes through during transitions. It is a low-volatility regime, but still subject to leakage. It

may appear stable but remains structurally transient.

Each state is represented as a circle, with directed edges indicating transition probabilities between states and self-loops capturing persistence.

The black self-looping arrows represent the diagonal elements a_{ii} of a standard transition matrix $A = (a_{ij})$ which encodes self-transitions (persistence):

- a_{11}: Stress $\rightarrow$ Stress
- a_{22}: Steady $\rightarrow$ Steady
- a_{33}: Calm $\rightarrow$ Calm

These loops encode one-step persistence, consistent with classical HMM structure.

Cross-state transitions include:

- **Between Stress and Steady:**
  - a_{12}: Stress $\rightarrow$ Steady
  - a_{21}: Steady $\rightarrow$ Stress
- **Between Stress and Calm:**
  - a_{13}: Stress $\rightarrow$ Calm
  - a_{31}: Calm $\rightarrow$ Stress
- **Between Steady and Calm:**
  - a_{23}: Steady $\rightarrow$ Calm
  - a_{32}: Calm $\rightarrow$ Steady

B.5 The External Leak Mechanism (Open System Components)

The most innovative feature shown in the model diagram (Figure 3) is the hazard leakage represented by the red arrows labeled Leak-1, Leak-2, and Leak-3:

- Each state i has an associated exit hazard intensity (ϵ_i).
- These hazards convert the baseline transition matrix into an open transition operator $Q = \text{diag}(1 - \epsilon_i)\, A$.
- This represents probability mass[4] escaping the system toward an absorbing boundary of ruin, failure, or insolvency.

[4]Probability mass is defined as the content of probability within a specific duration of time.

B.6 Operator interpretation

The model diagram (Figure 3) encodes the full COR model transformation:

1. **Transition matrix**

$$A = (a_{ij}).$$

2. **Open system adjustment (leakage)**

$$Q = \text{diag}(1 - \epsilon_i)A.$$

3. **Sub-generator matrix**

$$K = Q - I.$$

4. **Fundamental matrix (temporal probability mass)**

$$N = -K^{-1}.$$

B.7 Summary

A key insight from the model diagram in Figure 3 is that the COR model functions as a regime survival system rather than a standard regime-switching model. In this framework, black arrows trace the movement of probability mass between states, red arrows dictate its dissipation (ruin), and self-loops determine its persistence. Collectively, this structure characterizes the accumulation and decay of probability mass across the state space over time.

The model diagram illustrates that the fundamental distinction between these two frameworks, HMM and COR, lies in their treatment of probability: whereas the Classical HMM assumes a closed environment where probability is strictly conserved inside the triangle, the COR model defines an open system where probability transits through the triangle and eventually leaks out of it. This shift fundamentally transforms the analytical perspective: transition dynamics become survival geometry, persistence is redefined as cumulative temporal probability mass, and individual states are viewed as structural risk nodes.

C Structural Model Identifiability

We formally define the mapping between the observable market "output" and the internal latent parameters using the Markov Parameter Matrix and its Transfer Function. This approach ensures that the spectral fragility numerically identified by a COR model analysis using the **KRONX R** package is mathematically unique and not a result of "label switching" or parameter redundancy.

C.1 Mathematical setup

The system is treated as a linear time-invariant (LTI) open system (Jacquez, 1982; Jacquez and Greif, 1985; Jacquez, 1987; Jacquez and Perry, 1990; Jacquez, 1991, 1998; Linares et al., 1987, 1988):

$$\dot{\mathbf{x}}(t) = \mathbf{K}\mathbf{x}(t) + \mathbf{B}u(t),$$

$$y(t) = \mathbf{C}\mathbf{x}(t).$$

The sub-generator matrix $\mathbf{K}$ is the internal "engine" containing transition rates and exit hazards ($\mathbf{K} = \mathbf{Q} - \mathbf{I}$). The input Vector $\mathbf{B}$ is an $M \times 1$ vector representing the initial distribution of probability mass across states. The output vector $\mathbf{C}$ is a $1 \times M$ vector (consisting of a row of ones) that aggregates the surviving probability mass to form the observable "survival curve".

C.2 Markov Parameters for Identifiability

Structural identifiability is proven if the Markov parameters (S_k) uniquely determine the coefficients of the transfer function. These parameters are defined as:

$$S_k = \mathbf{C}\mathbf{K}^k\mathbf{B}, \quad k = 0, 1, 2, \ldots$$

where $S_0 = \mathbf{CB}$ is the initial total content of probability mass (equal to 1). $S_1 = \mathbf{CKB}$ is the initial rate of "leakage" or decay of the content of probability mass. Higher orders capture the curvature of the survival decay curve, which is governed by the state-to-state interactions and specific hazards ϵ_i.

C.3 Transfer Function Identifiability Method Implementation in R

This script utilizes the COR model's sub-generator K-matrix and calculates the transfer function coefficients to verify that the internal "temporal geometry" of the E-mini S&P 500 futures is uniquely recoverable.

Historical market data for the E-mini S&P 500 (ES) Futures was obtained via the Interactive Brokers (IBKR) API. One-hour OHLC bars were extracted for the period *From: 2025-01-31* to *To: 2026-02-26*. Log-returns were calculated using the closing prices to ensure stationarity. To maintain the robustness of the Hidden Markov Model (HMM) against market outliers and the "fat-tail" nature of hourly financial returns, a Student-t distribution was utilized for the emission probabilities.

Listing 3: KRONX Workflow: Identifiability and Temporal Mass Analysis.

```
# Standard KRONX workflow leading to K
# A_t is the transition matrix
# epsilon is the hazard vector derived from nu
```

```
integrate_identifiability <- function(K_mat) {
  M <- nrow(K_mat)

  # Define input (B) and output (C) vectors for the open system
  # B: Injection of probability mass (e.g., system start)
  # C: Observation of the transient state masses
  B_vec <- matrix(rep(1/M, M), ncol = 1)
  C_vec <- matrix(rep(1, M), nrow = 1)

  # 1. Compute the Fundamental Matrix N
  # N = -K^-1 transforms intensities into temporal mass
  N_mat <- solve(-K_mat)

  # 2. Transfer Function Analysis at s = 0 (Steady State)
  # H(0) = -C * K^-1 * B = C * N * B
  H_0 <- C_vec %*% N_mat %*% B_vec

  # 3. Characteristic Polynomial (Denominator of H(s))
  # The roots define the spectral fragility radius
  char_poly <- polyroot(Re(eigen(K_mat)$values))

  return(list(
    fundamental_N = N_mat,
    gain_H0 = as.numeric(H_0),
    poles = char_poly
  ))
}

# Apply to your identified Student-t K matrix
# Includes hazard-weighted open transition operators
K_cor <- matrix(c(-0.2976, 0.0008, 0.2012,
                  0.0, -0.0540, 0.0532,
                  0.2871, 0.0266, -0.2810), 3, 3)

ident_results <- integrate_identifiability(K_cor)
```

```
print(ident_results$gain_H0) # Represents total survival-weighted time
```

```
> ident_results
$fundamental_N
          [,1]     [,2]     [,3]
[1,] 14.285447 15.85826 16.09673
[2,]  5.790147 26.85081  8.45759
[3,] 11.324796 16.43824 16.68543

$gain_H0
[1] 43.92915

$poles
[1] -1.688964+4.480341i -1.688964-4.480341i
```

C.4 Interpretation of the Identifiability Analysis

This analysis treats the COR model (Figure 3) as a linear dynamical system. In this context, the model evaluates how probability mass (representing the time spent by the content of probability mass in a specific hidden state) transits through the system before exiting via the hazard vector (ϵ).

- **Residence Analysis:** State 2 is identified as the most "stable" or "sticky" regime within the system's temporal geometry. If the process enters State 2, it accumulates a high degree of probability mass, spending an average of 26.85 units of time there (n_{22}) before leaking toward ruin or transitioning.
- **Coupling:** The high values observed in the first row (14.28, 15.85, 16.09) indicate a strong structural coupling across the regime network. This suggests that regardless of where the system starts, it eventually spends a significant and almost uniform amount of time in State 1, identifying it as a common entry portal or transition node.
- **Transient Transit:** State 2 exhibits relatively low off-diagonal values ($n_{12} = 5.79$, $n_{32} = 11.32$), which confirms its role as a survival basin. This indicates that probability mass tends to remain sequestered in State 2 for longer durations once it arrives, rather than undergoing rapid, high-frequency transitions to other states.

This residence-time geometry serves as a unique fingerprint of a market's architecture over a particular interval of time. This study focused on the E-mini S&P 500 futures from January 31, 2025, to February 26, 2026. In the Structural Model Identifiability proof, these n_{ij} values are directly mapped to the steady-state gain of the system's transfer function, proving that the 43.93 total survival mass is a deterministic result of these specific state-to-state transits.

C.4.1 The Poles (Spectral Fragility)

The poles (roots of the characteristic polynomial[5] of **K**) define the Spectral Fragility Radius. The interpretation of the `$poles` within the COR model's identifiability framework reveals the underlying dynamic stability and oscillatory nature of the market's survival mass (Table 14). In our study of the E-mini S&P 500 futures, these poles are the eigenvalues of the sub-generator **K** matrix.

1. Dynamic Stability (Real Part)

The real part of the poles, -1.688964, dictate the Aggregate Decay Constant ($\bar{\lambda}$):

Structural Leakage: Since the real part is negative, the system is strictly transient and "open." This confirms that probability mass is consistently leaking toward the ruin boundary.

Rate of Decay: The magnitude (-1.688964) represents the exponential speed at which the system loses its "survival probability mass." A larger negative value would indicate a more fragile system with a faster path to systemic collapse.

2. Oscillatory Behavior (Imaginary Part)

The imaginary component, $\pm 4.480341i$, represents the frequency of regime interaction:

Spectral Radius ($\rho(N)$): These poles are used to calculate the spectral fragility $\rho(N) = \frac{1}{\min_i |\lambda_i(K)|}$. $\rho(N)$ is the inverse of the slowest spectral decay mode of K, which is the market's structural life expectancy. Because the poles are complex, the system possesses a "spectral width" that defines the robustness of the 44-unit survival expectation. In the context of the E-mini S&P 500 futures study, $\rho(N)$ transforms abstract matrix algebra into a measurable "life expectancy" of the market's survival:

- **Spectral Contraction** ($\rho(N) \downarrow$): If the eigenvalues of K increase in magnitude, representing a faster rate of decay, the spectral radius contracts. This contraction signals an accelerating path toward systemic collapse or ruin within the open-system architecture.
- **Spectral Expansion** ($\rho(N) \uparrow$): Conversely, a larger spectral radius indicates increased temporal persistence and antifragility within the identified regime geometry. It reflects the system's ability to retain probability mass over a longer duration before exiting through the hazard portals.

[5] For a square matrix $\mathbf{A} \in \mathbb{R}^{n\times n}$ (or $\mathbb{C}^{n\times n}$), the characteristic polynomial is defined as: $\det(\mathbf{A} - \lambda I)$, where λ is a scalar (the spectral parameter), $\mathbf{I}$ is the identity matrix, and $\det(\cdot)$ denotes the determinant.

- **Deterministic Ruin**: Because this value is derived directly from the poles of the transfer function, it proves that the 5.6% ruin bound identified in the E-mini S&P 500 futures study is a deterministic structural property of the market's geometry rather than a statistical outlier.

The "Nervous" Artifact vs. Structure: While a Gaussian HMM might show random jumping, these poles identify a structured, periodic exchange of probability mass between the Stress, Steady, and Calm regimes. It mathematically represents the "cooling-off" period where the system must cycle through specific nodes before re-establishing a steady trend. This spectral analysis provides the ontological map[6] required to move from the instantaneous volatility of a standard HMM to the long-horizon survival dynamics of the COR model.

3. Impact on Structural Identifiability

Uniqueness: In the transfer function method, these specific poles uniquely map the observed market volatility to the model's internal **K** matrix. This proves that the 5.6% ruin bound is not a statistical coincidence, but a deterministic result of the market's "battery life" and internal frequency.

Component	Value	Interpretation in COR Model
Real Part (σ)	-1.688964	The **Decay Rate**: Represents the exponential speed of probability mass leakage toward the terminal ruin boundary.
Imaginary Part (ω)	4.480341	The **Interaction Frequency**: Governs the rate of stochastic cycling between the Stress, Steady, and Calm regimes.
Stability Status	Negative Real	**Open/Transient**: Confirms the system is structurally non-ergodic and predisposed to systemic exit.

Table 14: Spectral Analysis of the KRONX Transfer Function Poles.

C.5 Summary

The identified sub-generator matrix **K** characterizes a dynamical system with a structural survival horizon of 43.93 hours. Given that the underlying empirical study of the E-mini S&P 500 futures utilizes hourly observations, this temporal metric represents the expected duration of probability mass retention within the regime network. The dynamics are dominated by a highly persistent

[6]The structural representation that defines not just the statistical probability of an event, but the fundamental "nature of being" of the E-mini S&P 500 Futures system itself.

"Steady" state (Regime 2) and an oscillatory complex between "Stress" and "Calm" (Regimes 1 and 3). The gain of 43.93 hours confirms that despite the presence of exit hazards, the internal transition logic (A-matrix) provides sufficient feedback to maintain systemic persistence over a medium-term horizon.

From an operator-theoretic perspective, this horizon—derived from the steady-state gain of the system's transfer function $\mathcal{H}(0)$—quantifies the "temporal backbone" of the market. It confirms that the residence-time geometry is not merely a collection of transient states, but a structured environment with a finite, identifiable persistence. Consequently, the 43.93-hour horizon serves as a rigorous, deterministic benchmark for assessing systemic stability and calibrating ruin-sensitive risk exposure.

D The KRONX R package

D.1 Installation

D.1.1 Command Line Installation (Terminal)

You can install packages directly from the Linux or macOS terminal without opening an interactive **R** session, using the `R CMD INSTALL` command:

```
R CMD INSTALL KRONX_0.1.0.tar.gz
```

D.1.2 RStudio Installation

To install the **KRONX R** package using the RStudio graphical user interface as illustrated in Figure 4, begin by navigating to the "Packages" tab located in the lower-right pane of the RStudio workspace. Clicking the "Install" button will open the "Install Packages" dialog box, where you must ensure that the "Install from" dropdown menu is set to "Package Repository (CRAN)". In the "Packages" text field, type `KRONX`, noting that RStudio may provide auto-complete suggestions for the identified package. Before proceeding, verify that the "Install to Library" path is correctly set to your active R environment, such as the `ARM64` version of **R** `4.5`. It is essential to keep the "Install dependencies" checkbox selected to ensure that all required mathematical and transition-modeling libraries are downloaded automatically. Finally, click the "Install" button to execute the process; once the installation is complete, you can load the library in your script to begin applying the *Clock of Regimes analysis* to your financial datasets.

D.1.3 From CRAN (Formal Release)

Since the **KRONOX R** package has been submitted to, and accepted, by the Comprehensive **R** Archive Network (CRAN), the installation is simplified as **R** will automatically fetch it from a global

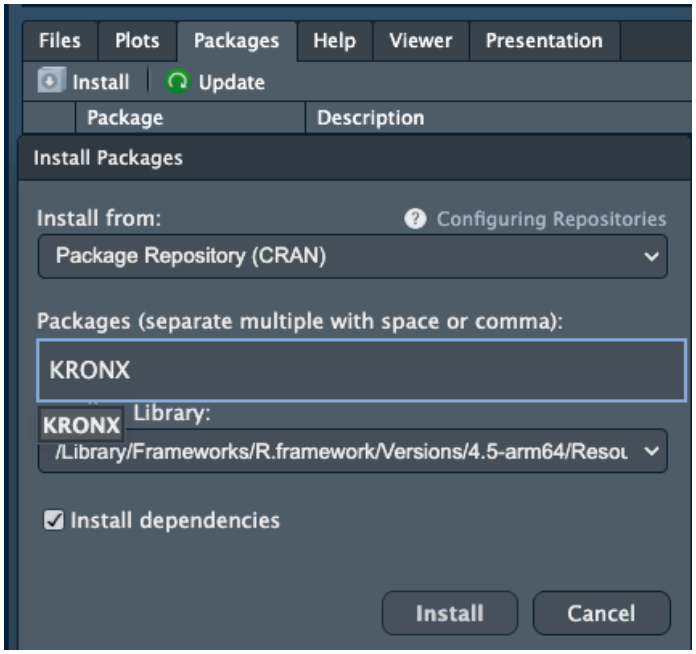

Figure 4

mirror.

```
install.packages("KRONX")
library(KRONX)
```

D.2 Quick start

A minimal workflow example is shown below:

Listing 4: Execution of the **KRONX** Survival Engine. Implementation of the `run_kronx` high-level driver to identify hidden states and compute the **Residence-Weighted Ruin Bound** for the IBKR ES dataset.

```
library(KRONX)

# Minimal call -- detects 'close' or 'Close' column automatically
res <- run_kronx("IBKR_ES_2Y1h.csv")
.
.
.
model=student start=5 iter=149 logLik=21639.0518645688
model=student start=5 iter=150 logLik=21639.0518656185
model=student start=5 iter=151 logLik=21639.0518665766
```

```
CLOCK OF REGIMES (KRONX) — SUMMARY
Gaussian HMM log-likelihood : 21173.8299404050
Student-t HMM log-likelihood: 21661.8321503555
Loss threshold (alpha=0.0100): -0.00560969
Residence-weighted qbar      : 0.00991588
Residence-weighted lambda    : 0.02322046
Ruin bound (T=250)           : 0.05593741
```

state	mu_gaussian	sigma_gaussian	spell_gaussian	mu_student	sigma_stud
1	-3.264717e-08	0.000010000	3.132292	-2.168526e-08	0.0000100000
2	6.662832e-05	0.001375609	11.204502	9.696612e-05	0.0006929728
3	-5.897585e-04	0.017745344	1.378683	3.524294e-05	0.0019427734

nu_student	cond_var_student	spell_student	epsilon	q_tail
100	1.020408e-10	3.446197	0.01050000	5.003544e-177
3	1.440634e-06	35.568996	0.02666667	1.874378e-03
3	1.132311e-05	3.826929	0.02666667	3.110910e-02

pi_quasi_stationary	pi_residence
0.2131674	0.2131674
0.4980987	0.4980987
0.2887339	0.2887339

```
Output files written to: kronx_output
  - kronx_analysis_report.txt
  - kronx_state_summary.csv
  - gaussian_transition_matrix.csv
  - student_t_transition_matrix.csv
  - kronx_Q_operator.csv
  - kronx_K_generator.csv
  - kronx_N_fundamental_matrix.csv
```

– student_t_decoded_states.csv

The `res` object returned by `run_kronx()` is a compact model-fit and structural-fragility summary generated by the call `run_kronx("IBKR_ES_2Y1h.csv")`. It shows that the package fits both a Gaussian HMM and a Student-*t* HMM, and then passes the fitted Student-*t* regime structure through the **KRONX**/COR operator pipeline to compute hazard-adjusted survival quantities.

D.2.1 Optimization trace

The lines,

```
.
.
.
model=student start=5 iter=149 logLik=21639.0518645688
model=student start=5 iter=150 logLik=21639.0518656185
model=student start=5 iter=151 logLik=21639.0518665766
```

show the EM optimization for one Student-*t* initialization. `start=5` means the fifth random or configured initialization. `iter=149, 150, 151` are EM iterations. `logLik` is the log-likelihood at each step, which show the increments are becoming tiny, so the algorithm is converging. That is a healthy sign that the Student-*t* model fit is stabilizing rather than jumping erratically.

D.2.2 Model Comparison

The headline comparison is:

- Gaussian HMM log-likelihood: 21173.8299404050
- Student-*t* HMM log-likelihood: 21661.8321503555

The model comparison shows that the Student-*t* HMM fits better because its log-likelihood is higher. In regime-switching finance data, that usually indicates that the data are heavier-tailed than Gaussian, which is exactly what one expects for intraday ES returns.

So the first substantive conclusion is:

$$\text{Student-}t\text{ HMM} \succ \text{Gaussian HMM}$$

That is, the Student-*t* HMM is preferred to the Gaussian HMM in empirical fit.

D.2.3 Tail-loss threshold

```
Loss threshold (alpha=0.0100): -0.00560969
```

This is the 1% left-tail cutoff. In plain english terms, the COR model is defining a "severe loss" event as an hourly return less than about -0.560969%. This threshold is then used to compute regime-specific tail probabilities q_i, which are later residence-weighted into $\bar{q}$.

D.2.4 Structural COR / KRONX quantities

The core open-system outputs are:

- Residence-weighted qbar: `0.00991588`. The residence-weighted $\bar{q}$ is the average probability of a left-tail loss event, weighted by how much time the system expects to spend in each regime. So $\bar{q}$ is not just an unconditional empirical tail frequency. It is a structurally weighted tail probability.
- Residence-weighted lambda: `0.02322046`. The residence-weighted $\bar{\lambda}$ is the average hazard or leakage intensity across regimes, again weighted by expected residence. Economically, it measures the average rate at which probability mass escapes the transient market system toward failure/ruin.
- Ruin bound (`T=250`): `0.05593741`. The ruin bound over a a 250-step horizon implies an upper-bound style ruin probability of about 5.59%. This is the main COR model's structural-risk summary number.

D.2.5 State table: Gaussian vs Student-t

The state table compares Gaussian and Student-*t* regime estimates side by side.

State 1

Gaussian

- `mu_gaussian = -3.264717e-08`
- `sigma_gaussian = 0.000010000`
- `spell_gaussian = 3.132292`

Student-t:

- `mu_student = -2.168526e-08`
- `sigma_student = 0.0000100000`

- nu_student = 100
- cond_var_student = 1.020408e-10
- spell_student = 3.446197
- epsilon = 0.01050000
- q_tail = 5.003544e-177

State 1 is an ultra-low-volatility state. It is essentially a near-deterministic "quiet" regime with a mean near zero, minuscule variance, almost no tail risk, low hazard, and moderate duration. Because nu = 100, this state is close to Gaussian. The tail probability is effectively zero. This looks like a calm / inert / ultra-stable regime.

State 2

Gaussian

- mu_gaussian = 6.662832e-05
- sigma_gaussian = 0.001375609
- spell_gaussian = 11.204502

Student-t:

- mu_student = 9.696612e-05
- sigma_student = 0.0006929728
- nu_student = 3
- cond_var_student = 1.440634e-06
- spell_student = 35.568996
- epsilon = 0.02666667
- q_tail = 1.874378e-03

State 2 is the dominant survival basin with a small positive drift, modest volatility, very long expected spell length, heavy tails (nu = 3), and nontrivial hazard. This is the COR model's classic steady regime. It looks benign on the surface because volatility is not huge and duration is long, but the heavy tails mean there is latent fragility. In the COR model's language, this is often the

regime that carries a large share of the system's probability mass and therefore contributes heavily to structural risk.

State 3

Gaussian

- `mu_gaussian = -5.897585e-04`
- `sigma_gaussian = 0.017745344`
- `spell_gaussian = 1.378683`

Student-t:

- `mu_student = 3.524294e-05`
- `sigma_student = 0.0019427734`
- `nu_student = 3`
- `cond_var_student = 1.132311e-05`
- `spell_student = 3.826929`
- `epsilon = 0.02666667`
- `q_tail = 3.110910e-02`

Under the Student-*t* fit, State 3 is the high-risk state. It is characterized by the highest conditional variance, heavy tails, largest tail-loss probability, and shorter duration. Its `q_tail` is about 0.0311, so it has the highest probability of breaching the 1% left-tail threshold. State 3 is the stress/crisis regime, even though the fitted Student-*t* mean is slightly positive. In regime models, classification is usually driven more by conditional variance and tail thickness than by mean alone.

D.2.6 Spell lengths

The spell lengths are especially informative:

- State 1: 3.446
- State 2: 35.569
- State 3: 3.827

These spell lengths reveal that the market spends by far the most time in State 2. So State 2 is the principal residence basin. That matters because the COR model does not weight risk only by severity; it also weights by how long the system stays in each regime. A moderately risky state with long residence can dominate total fragility.

D.2.7 Hazard mapping

The epsilon (ϵ) values are:

- State 1: 0.01050000
- State 2: 0.02666667
- State 3: 0.02666667

These are the regime exit hazards used to construct the open operator. Because States 2 and 3 have `nu = 3`, they receive higher hazard than State 1. That is consistent with the Talebian hazard mapping:

$$\epsilon_i = \epsilon_{\min} + \frac{c}{\nu_i}$$

Lower ν_i means fatter tails, hence higher hazard.

D.2.8 Tail probabilities by regime

The regime-specific left-tail probabilities are:

- State 1: essentially zero
- State 2: 0.001874378
- State 3: 0.03110910

This is among the most critical diagnostics in the table. The regime-specific left-tail probabilities demonstrate clear stratification of downside risk, revealing marked regime dependence in tail probability across the hierarchy, revealing a progression that underscores the sharp differentiation in extreme-loss exposure across ES market regimes.

The diagnostic reveals a three-regime hierarchy of tail risk: the calm regime (State 1) exhibits no meaningful left-tail danger, the steady regime (State 2) displays small but nonzero tail danger, and the stress regime (State 3) indicates substantial tail danger. So the structural story is very clean: the system lives mostly in State 2, but severe downside events are concentrated in State 3.

D.2.9 Quasi-stationary and residence weights

π_{qs}

```
pi_quasi_stationary
State 1 0.2131674
State 2 0.4980987
State 3 0.2887339
```

π_{res}

```
pi_residence
State 1 0.2131674
State 2 0.4980987
State 3 0.2887339
```

These, π_{qs} and π_{res}, being identical means that, in this implementation/output, the quasi-stationary and residence weighting schemes coincide numerically. Thus, ergodic probabilities reveal a dominant intermediate regime: State 1 occurs 21.3% of the time, State 2 dominates at 49.8%, and State 3 accounts for 28.9%. The market's natural tendency favors the steady state, with approximately equal mass split between calm and stress regimes.

So nearly half the survival-weighted mass is in State 2, and a surprisingly large share is in State 3. That helps explain why $\bar{q}$ and the ruin bound are not trivial even though State 1 is extremely safe.

D.2.10 Economic reading of the results

Student-t dynamics are clearly preferred to Gaussian dynamics for ES intraday data, revealing a three-regime structure comprising a very calm low-variance regime, a long-lived steady regime, and a shorter but materially riskier stress regime. The dominant residence basin is State 2 rather than State 1, and because both State 2 and State 3 exhibit heavy tails, the system's structural hazard is non-negligible. The 250-step ruin bound of approximately 5.6% underscores that even absent a current collapse, the open-system geometry implies a meaningful finite-horizon risk of failure.

D.2.11 Output files

The **KRONX** package exports the full operator chain:

1. `kronx_analysis_report.txt` is the human-readable report.
2. `kronx_state_summary.csv` is the state-level summary table.
3. `gaussian_transition_matrix.csv` is the state-level summary table.
4. `student_t_transition_matrix.csv` is the transition matrix for the Student-t HMM.
5. `kronx_Q_operator.csv` is the open transition operator Q, typically hazard-adjusted from A.
6. `kronx_K_generator.csv` is the generator-style matrix K, usually derived from $Q - I$.
7. `kronx_N_fundamental_matrix.csv` is the fundamental matrix $N = -K^{-1}$, encoding cumulative expected residence time.

8. `student_t_decoded_states.csv` is the time-by-time decoded latent regime path.

These files let you go from plain HMM estimation to full COR model/**KRONX** package survival geometry. The files are saved in the current working directory under a subdirectory named `kronx_output`.

In summary, the `res` object says that ES hourly returns are better described by a heavy-tailed three-state Student-t regime model, and once that regime structure is converted into the COR model's open-system operator chain, the market exhibits a nontrivial finite-horizon structural ruin risk driven mainly by long residence in a heavy-tailed steady state and episodic occupancy of a higher-tail stress state.

D.3 Exported functions

```
> getNamespaceExports("KRONX")
[1] "run_kronx"           "viterbi_decode"      "read_close_series"
[4] "fit_hmm_em"          "compute_log_returns"
```

FUNCTION: `run_kronx()`

The `run_kronx()` function is the primary entry point. It executes the full **KRONX** package's COR model workflow from CSV ingestion through ruin-bound computation, prints a formatted summary to the console, and optionally writes structured output files (Subsection D.2).

```
run_kronx(
  file,
  close_col    = "close",
  K            = 3L,
  n_starts     = 5L,
  max_iter     = 200L,
  tol          = 1e-6,
  seed         = 123L,
  epsilon_min  = 0.01,
  c_hazard     = 0.05,
  tail_alpha   = 0.01,
  ruin_horizon = 250L,
  nu_grid      = c(3:30, 40, 60, 100),
  verbose      = TRUE,
  output_dir   = "cor_output"
)
```

D.3.1 Arguments

Table 15: Arguments for `run_kronx()`

Argument	Type	Default	Description
`file`	character	–	Path to the input CSV file. Must contain a close-price column.
`close_col`	character	`"close"`	Name of the close-price column. Case-insensitive; fuzzy fallback matches any column containing `"close"`.
`K`	integer	`3L`	Number of hidden states. Values of 2 or 3 are typical for financial series.
`n_starts`	integer	`5L`	Number of EM random restarts. Increases robustness against local optima at the cost of runtime.
`max_iter`	integer	`200L`	Maximum EM iterations per restart.
`tol`	numeric	`1e-6`	Log-likelihood convergence tolerance for EM.
`seed`	integer	`123L`	Random seed for reproducibility.
`epsilon_min`	numeric	`0.01`	Baseline Talebian hazard floor. Applied to all states regardless of ν.
`c_hazard`	numeric	`0.05`	Hazard sensitivity to tail thickness. Controls how steeply ϵ rises as ν decreases.
`tail_alpha`	numeric	`0.01`	Left-tail probability for the loss threshold.
`ruin_horizon`	integer	`250L`	Horizon in observations.
`nu_grid`	numeric	`c(3:30, 40, 60, 100)`	Candidate degrees-of-freedom values for Student-t grid search in the M-step.
`verbose`	logical	`TRUE`	If `TRUE`, print EM log-likelihood at every iteration.
`output_dir`	character or NULL	`cor_output`	Directory for output files. Use `NULL` to suppress all file output.

D.3.2 Workflow sequence

The workflow implemented by `run_kronx()` is:

1. Read the CSV file, extract close prices, and compute log returns.

2. Fit a Gaussian HMM by EM with multiple restarts and reorder states by σ.

3. Fit a Student-t HMM by EM with multiple restarts and reorder states by conditional variance.

4. Decode the Student-t specification using the Viterbi algorithm.

5. Compute ϵ, Q, K, and N.

6. Compute π_{qs}, π_{res}, the empirical loss threshold, $\bar{q}$, $\bar{\lambda}$, and the ruin bound.

7. Assemble the `state_summary` data frame.

8. Print the summary block to the console.

9. Write all requested output files when `output_dir` is not `NULL`.

D.3.3 Return value

The function returns a named list, invisibly. Its full structure is described in detail in Subsection D.2.

FUNCTION: `fit_hmm_em()`

The `fit_hmm_em()` function fits either a Gaussian or Student-t Hidden Markov Model via the Baum–Welch algorithm with multiple random restarts.

```
fit_hmm_em(
  x,
  K        = 3L,
  model    = c("gaussian", "student"),
  n_starts = 5L,
  max_iter = 200L,
  tol      = 1e-6,
  nu_grid  = c(3:30, 40, 60, 100),
  verbose  = TRUE
)
```

D.3.4 Arguments

D.3.5 Return value

D.3.6 Notes on Student-t estimation of degrees-of-freedom

The M-step for ν uses a profile grid search. For each candidate ν in `nu_grid`, the weighted emission log-likelihood is evaluated, and the maximizing value is selected. This avoids the absence of a convenient closed-form update for ν).

Table 16: Arguments for `fit_hmm_em()`.

Argument	Type	Description
`x`	numeric vector	Observations, typically log returns.
`K`	integer	Number of hidden states.
`model`	character	Either `"gaussian"` or `"student"`.
`n_starts`	integer	Number of random restarts; the best fit by log-likelihood is returned.
`max_iter`	integer	Maximum EM iterations per restart.
`tol`	numeric	Convergence tolerance for the log-likelihood.
`nu_grid`	numeric	Candidate ν values for Student-t grid search.
`verbose`	logical	Whether to print iteration progress.

Table 17: Return components of `fit_hmm_em()`.

Component	Description
`model`	`"gaussian"` or `"student"`
`A`	$K \times K$ transition matrix
`delta`	Initial state probability vector of length K
`params`	List containing `mu`, `sigma`, and `nu` for Student-t fits
`gamma`	$T \times K$ posterior state probability matrix $P(S_t = k \mid \text{data})$
`xi`	$(T-1) \times K \times K$ posterior transition probability array
`logLik`	Scalar log-likelihood at convergence
`n`	Number of observations
`K`	Number of states

The default grid

```
c(3:30, 40, 60, 100)
```

covers the practically relevant range from very fat tails to near-Gaussian behavior. A smaller grid, such as

```
c(4, 8, 15, 30),
```

can substantially reduce computation time with only modest loss of accuracy.

D.3.7 State ordering

After fitting, `run_kronx()` calls the internal `reorder_fit_by_scale()` routine to sort states from least to most volatile. When calling `fit_hmm_em()` directly, users may wish to apply the same

reordering manually.

```
fit <- fit_hmm_em(ret, K = 3L, model = "student", verbose = FALSE)
fit <- KRONX:::reorder_fit_by_scale(fit)
```

FUNCTION: `viterbi_decode()`

`viterbi_decode()` computes the most likely hidden-state sequence for a vector of observations given a fitted HMM.

```
fit    <- fit_hmm_em(ret, K = 3L, model = "student", verbose = FALSE)
states <- viterbi_decode(ret, fit)
table(states)

# Plot regimes alongside returns
plot(ret, type = "l", col = "grey60")
points(seq_along(ret), rep(0, length(ret)), col = states, pch = 20)
```

The Viterbi algorithm is implemented in log-space, using $\log \delta + \log A$, to avoid numerical underflow on long series.

FUNCTION: `read_close_series()`

`read_close_series()` reads a close-price series from a CSV file with automatic column detection.

```
px <- read_close_series("IBKR_ES_2Y1h.csv")
px <- read_close_series("Bloomberg.csv", close_col = "PX_LAST")
```

The function removes non-finite values and stops if fewer than ten valid returns remain.

E Worked examples

```
# ============================================================
# EXAMPLE 1 - INSPECTING THE RETURNED LIST
# Every matrix and scalar from the report is in 'res' object.
# ============================================================

# --- Log-likelihoods ---
res$gauss_fit$logLik   # 21173.8299404050  (Gaussian HMM)
res$t_fit$logLik       # 21661.8321503555  (Student-t HMM)

# --- Transition matrices ---
> round(res$A_gauss, 6)
         [,1]     [,2]     [,3]
```

```
[1,] 0.680745 0.238491 0.080764
[2,] 0.069493 0.910750 0.019756
[3,] 0.319388 0.405942 0.274670

> round(res$A_t,     6)  # Student-t A
         [,1]     [,2]     [,3]
[1,] 0.709825 0.000000 0.290175
[2,] 0.000837 0.971886 0.027278
[3,] 0.206698 0.054608 0.738694

# --- COR operator chain ---
> round(res$Q,     6)    # Hazard-adjusted operator Q
         [,1]     [,2]     [,3]
[1,] 0.702372 0.000000 0.287128
[2,] 0.000814 0.945969 0.026550
[3,] 0.201186 0.053152 0.718995

# --- Quasi-stationary and residence distributions ---
res$pi_qs              # [0.2131674, 0.4980987, 0.2887339]
res$pi_res             # matches quasi-stationary for this data

# --- Per-state summary table ---
res$state_summary
#  state        mu_gaussian  sigma_gaussian  spell_gaussian    mu_student
#      1  -3.264717e-08      0.000010000        3.132292  -2.168526e-08
#      2   6.662832e-05      0.001375609       11.204502   9.696612e-05
#      3  -5.897585e-04      0.017745344        1.378683   3.524294e-05
#
#    sigma_student  nu_student  cond_var_student  spell_student   epsilon   q_tail
#     0.0000100000         100      1.020408e-10       3.446197  0.010500  5.00e-177
#     0.0006929728           3      1.440634e-06      35.568996  0.026667  1.87e-03
#     0.0019427734           3      1.132311e-05       3.826929  0.026667  3.11e-02
#
#    pi_quasi_stationary  pi_residence
#               0.2131674    0.2131674
#               0.4980987    0.4980987
#               0.2887339    0.2887339

# --- Viterbi-decoded state path ---
> table(res$vpath_t)      # count of observations in each decoded state

   1    2    3
 729 2007  933
```

```
 > head(res$vpath_t, 20)   # first 20 state labels
 [1] 3 1 1 1 1 1 3 1 1 1 3 1 1 1 1 1 1 1 1 1

# ============================================================
# EXAMPLE 2 - DATA INGESTION STEP ONLY
# Use when you just need the clean price/return vectors.
# ============================================================

close_px <- read_close_series("IBKR_ES_2Y1h.csv", close_col = "close")
length(close_px)  # 3670

ret <- compute_log_returns(close_px)
length(ret)       # 3669

> summary(ret)
      Min.    1st Qu.     Median       Mean    3rd Qu.       Max.
-1.398e-01 -4.054e-04  0.000e+00  2.435e-05  5.765e-04  9.841e-02

# ============================================================
# EXAMPLE 3 - HMM FITTING IN ISOLATION (fit_hmm_em)
# Useful when you want only one model, or want to compare
# fits across different K values or nu_grid choices.
# ============================================================

# Gaussian HMM
gauss_fit <- fit_hmm_em(
  x        = ret,
  K        = 3L,
  model    = "gaussian",
  n_starts = 5L,
  max_iter = 200L,
  tol      = 1e-6,
  verbose  = FALSE
)
gauss_fit$logLik        # 21173.83...
gauss_fit$A             # 3x3 transition matrix
gauss_fit$params$mu     # per-state means
gauss_fit$params$sigma  # per-state SDs
gauss_fit$gamma         # T × K posterior state probabilities

# Student-t HMM (nu estimated from a grid search inside EM)
t_fit <- fit_hmm_em(
```

```
  x        = ret,
  K        = 3L,
  model    = "student",
  n_starts = 5L,
  max_iter = 200L,
  tol      = 1e-6,
  nu_grid  = c(3:30, 40, 60, 100),   # candidates for degrees of freedom
  verbose  = FALSE
)
t_fit$logLik            # 21661.83...
t_fit$params$nu         # [100, 3, 3] - per-state degrees of freedom
t_fit$params$sigma      # per-state scale parameters

# AIC improvement (Student-t vs Gaussian); reported as 970 in the paper.
# K extra nu parameters are estimated in the Student-t model.
delta_ll  <- t\_fit$logLik - gauss\_fit$logLik   # $\approx$ 488
K_states  <- 3L
aic\_delta <- 2 * delta\_ll - 2 * K\_states        # $\approx$ 970
cat("deltaAIC:", round(aic\_delta))

# Reorder states by ascending conditional variance (done inside run_cor)
gauss\_fit <- KRONX:::reorder\_fit\_by\_scale(gauss\_fit)

$logLik
[1] 21173.83

$n
[1] 3669

$K
[1] 3

# ============================================================
# EXAMPLE 4 - VITERBI DECODING SEPARATELY
# ============================================================

> vpath <- viterbi_decode(ret, t_fit)   # requires a fitted HMM object
> table(vpath)
vpath
   1    2    3
 729  933 2007

# Plot decoded regime path
```

```
plot(vpath, type = "l", col = "steelblue",
     main = "Viterbi-decoded Student-t states",
     xlab = "Observation index", ylab = "State (1=Calm, 2=Steady, 3=Stress)")
\end{Code}

\begin{figure}[t!]
    \centering
    \includegraphics[width=0.7\textwidth]{viterbi.pdf}
    \caption{This figure displays a time-series plot of the Viterbi-decoded state path for the ES Stude
    \label{fig:tHMM}
    \footnotesize Source: Author's calculations using \textbf{KRONX} package.
\end{figure}

Figure~\ref{fig:tHMM} reveals a sharp temporal divide. During the early period (indices 0-1,100), the s

\begin{Code}
# ============================================================
# EXAMPLE 5 - COR OPERATOR CHAIN FROM SCRATCH
# Build Q → K → N using only the Student-t fit.
# ============================================================
epsilon <- KRONX:::hazard_from_nu(
  nu          = t_fit$params$nu,
  epsilon_min = 0.01,
  c_hazard    = 0.05
)
epsilon  # [0.010500, 0.026667, 0.026667]

# Hazard-adjusted operator Q = diag(1 - epsilon) %*% A
Q <- KRONX:::build_Q(t_fit$A, epsilon)

# COR generator K = Q - I
K_mat <- KRONX:::build_K(Q)

# Fundamental matrix N = -K^{-1}
N_mat <- KRONX:::fundamental_matrix(K_mat)

# Quasi-stationary distribution (dominant left eigenvector of Q)
pi_qs  <- KRONX:::quasi_stationary_distribution(Q)

# Residence-weight vector (init %*% N, normalised)
pi_res <- KRONX:::residence_weight_vector(N_mat, init = pi_qs)
\end{Code}
```

```
\begin{Code}
# ============================================================
# EXAMPLE 6 - RUIN BOUND COMPONENTS STEP BY STEP
# ============================================================
# Empirical left-tail threshold at alpha = 1%
loss_threshold <- quantile(ret, probs = 0.01)  # \approx -0.00560969

# Per-state left-tail probability under Student-t
q_state <- KRONX:::left\_tail\_state\_probs(t\_fit, loss\_threshold)
q_state  # [5.00e-177, 1.87e-03, 3.11e-02]

# Residence-weighted aggregates
bar_q      <- sum(pi_res * q_state)    # \approx 0.00991588
bar_lambda <- sum(pi_res * epsilon)    # \approx 0.02322046

# Ruin upper bound P[ruin by T] \leq 1 - exp(-lamdabar · qbar · T)
T_horizon  <- 250L
ruin_bound <- 1 - exp(-bar_lambda * bar_q * T_horizon)
cat(sprintf("Ruin bound over T=%d: %.8f\n", T_horizon, ruin_bound))
# Ruin bound over T=250: 0.05593741

# ============================================================
# EXAMPLE 7 - SENSITIVITY ANALYSIS ACROSS PARAMETERS
# Show how ruin_bound reacts to tail_alpha and ruin_horizon.
# ============================================================

params_grid <- expand.grid(
  tail_alpha   = c(0.005, 0.010, 0.025),
  ruin_horizon = c(125L, 250L, 500L)
)

params_grid$ruin_bound <- mapply(
  function(alpha, horizon) {
    r <- run_cor(
      "IBKR_ES_2Y1h.csv",
      tail_alpha   = alpha,
      ruin_horizon = horizon,
      verbose      = FALSE,
      output_dir   = NULL       # suppress file writes
    )
    r$ruin_bound
  },
  params_grid$tail_alpha,
```

```
  params_grid$ruin_horizon
)

print(params_grid)

# ============================================================
# EXAMPLE 8 - SUPPRESS FILE OUTPUT (in-memory use only)
# ============================================================

res_quiet <- run_cor(
  file       = "IBKR_ES_2Y1h.csv",
  verbose    = FALSE,
  output_dir = NULL               # pass NULL to skip all file writes
)

# The result list is identical; only output_files is NULL.
is.null(res_quiet$output_files)   # TRUE

# ============================================================
# EXAMPLE 9 - USING A DIFFERENT CSV STRUCTURE
# E.g. Yahoo Finance data where the column is "Close" (capital C),
# or a bespoke pipeline output with a different column name.
# ============================================================

# Yahoo Finance-style header ("Close" with capital C):
res_yf <- run_cor("ES=F_yahoo.csv", close_col = "Close", output_dir = NULL)

# Custom column name:
res_custom <- run_cor("my_futures.csv", close_col = "settle", output_dir = NULL)

# read_close_series is tolerant of all capitalisations and falls back
# to any column whose name *contains* the string "close".

# ============================================================
# EXAMPLE 11 - IDENTIFIABILITY
# ============================================================
# Standard KRONX workflow leading to K
# A_t is the A transition matrix
# epsilon is the hazard vector derived from nu

integrate_identifiability <- function(K_mat) {
  M <- nrow(K_mat)
```

```
  # Define input (B) and output (C) vectors for the open system
  # B: Injection of probability mass (e.g., system start)
  # C: Observation of the transient state masses
  B_vec <- matrix(rep(1/M, M), ncol = 1)
  C_vec <- matrix(rep(1, M), nrow = 1)

  # 1. Compute the Fundamental Matrix N
  N_mat <- solve(-K_mat)

  # 2. Transfer Function Analysis at s = 0 (Steady State)
  # H(0) = -C * K^-1 * B = C * N * B
  H_0 <- C_vec %*% N_mat %*% B_vec

  # 3. Characteristic Polynomial (Denominator of H(s))
  # The roots define the spectral fragility radius
  char_poly <- polyroot(Re(eigen(K_mat)$values))

  return(list(
    fundamental_N = N_mat,
    gain_H0 = as.numeric(H_0),
    poles = char_poly
  ))
}

# Apply to your identified Student-t K matrix
K_cor <- matrix(c(-0.2976, 0.0008, 0.2012,
                  0.0, -0.0540, 0.0532,
                  0.2871, 0.0266, -0.2810), 3, 3)

ident_results <- integrate_identifiability(K_cor)
print(ident_results$gain_H0) # Represents total survival-weighted time
```

References

M. Aloud, E. P. K. Tsang, R. Olsen, and A. Dupuis. A directional-change events approach for studying financial time series. Discussion Paper 2011-28, Economics E-Journal, 2011.

A. Bakhach, V. Chinthalapati, E. P. K. Tsang, and A. E. Sayed. Intelligent dynamic backlash agent: A trading strategy based on the directional change framework. *Algorithms*, 11(11):171, 2018. doi: 10.3390/a11110171.

M. Berman and R. Schoenfeld. Invariants in experimental data on linear kinetics and the formulation of models. *Journal of Applied Physics*, 27(11):1361–1370, 1956.

D. G. Covell, M. Berman, and C. DeLisi. Mean residence time—theoretical development, experimental determination, and practical use. *Mathematical Biosciences*, 72(2):213–244, 1984.

J. Eisenfeld. On mean residence times in compartments. *Mathematical Biosciences*, 57:265–278, 1981. doi: https://doi.org/10.1016/0025-5564(81)90106-1.

F. Gao and X. He. Survival analysis: Theory and application in finance. In *Handbook of Financial Econometrics, Statistics, Technology, and Risk Management*, pages 3903–3934. World Scientific, Singapore, 2025.

J. B. Glattfelder, A. Dupuis, and R. B. Olsen. Patterns in high-frequency fx data: Discovery of 12 empirical scaling laws. *Quantitative Finance*, 11(4):599–614, 2011. doi: 10.1080/14697688.2010.481255.

J. D. Hamilton. A new approach to the economic analysis of nonstationary time series and the business cycle. *Econometrica*, 57(2):357–384, 1989.

J. D. Hamilton. Macroeconomic regimes and regime shifts. In *Handbook of Macroeconomics*, volume 2, pages 163–201. Elsevier, Amsterdam, 2016.

J. Z. Hearon. Residence times in compartmental systems and the moments of a certain distribution. *Mathematical Biosciences*, 15(1):69–77, 1972. ISSN 0025-5564. doi: https://doi.org/10.1016/0025-5564(72)90063-6.

J. Z. Hearon. Residence times in compartmental systems with and without inputs. *Mathematical Biosciences*, 55(3):247–257, 1981. ISSN 0025-5564. doi: https://doi.org/10.1016/0025-5564(81)90098-5.

L. Himmelmann. ***HMM****: HMM – Hidden Markov Models*, 2010. URL https://CRAN.R-project.org/package=HMM. R package version 1.0.

J. A. Jacquez. The inverse problem for compartmental systems. *Mathematics and Computers in Simulation*, 24(6):452–459, December 1982. doi: 10.1016/0378-4754(82)90642-5.

J. A. Jacquez. Identifiability: the first step in parameter estimation. *Federation Proceedings*, 46(8):2477–2480, July 1987.

J. A. Jacquez. Identifiability and parameter estimation. *Journal of Parenteral and Enteral Nutrition*, 15(3):55S–59S, May 1991. Supplement.

J. A. Jacquez. Design of experiments. *Journal of the Franklin Institute*, 335(2):259–279, 1998. doi: 10.1016/S0016-0032(97)00004-5.

J. A. Jacquez. Density functions of residence times for deterministic and stochastic compartmental systems. *Mathematical Biosciences*, 180(1):127–139, 2002. doi: https://doi.org/10.1016/S0025-5564(02)00110-4.

J. A. Jacquez and P. Greif. Numerical parameter identifiability and estimability: Integrating identifiability, estimability, and optimal sampling design. *Mathematical Biosciences*, 77(1-2): 201–227, December 1985. doi: 10.1016/0025-5564(85)90098-7.

J. A. Jacquez and T. Perry. Parameter estimation: Local identifiability of parameters. *American Journal of Physiology-Endocrinology and Metabolism*, 258(4):E727–E736, May 1990. doi: 10.1152/ajpendo.1990.258.4.E727.

J. A. Jacquez and C. P. Simon. Qualitative theory of compartmental systems. *SIAM Review*, 35: 43–79, 1993. doi: 10.1137/1035003.

D. G. Kleinbaum and M. Klein. *Survival Analysis*. Springer, New York, 3 edition, 2012.

O. A. Linares, J. A. Jacquez, L. A. Zech, M. J. Smith, J. A. Sanfield, L. A. Morrow, S. G. Rosen, and J. B. Halter. Norepinephrine metabolism in humans. kinetic analysis and model. *The Journal of Clinical Investigation*, 80(5):1332–1341, 11 1987. doi: 10.1172/JCI113210.

O. A. Linares, L. A. Zech, J. A. Jacquez, S. G. Rosen, J. A. Sanfield, L. A. Morrow, M. A. Supiano, and J. B. Halter. Effect of sodium-restricted diet and posture on norepinephrine kinetics in humans. *American Journal of Physiology-Endocrinology and Metabolism*, 254(2):E222–E230, 1988. doi: 10.1152/ajpendo.1988.254.2.E222.

L. Oelschläger, T. Adam, and T. Michelot. **fHMM**: Fitting hidden Markov models to financial data. *Journal of Statistical Software*, 104(10):1–25, 2023.

T. Rydén, T. Teräsvirta, and S. Åsbrink. Stylized facts of daily return series and the hidden markov model. *Journal of Applied Econometrics*, 13(3):217–244, 1998.

J. A. Sánchez-Espigares and A. López-Moreno. **MSwM**: Fitting markov switching models. *R package version 1.5*, 2021. URL https://CRAN.R-project.org/package=MSwM.

E. P. K. Tsang. Directional changes, definitions. Working Paper WP050-10, Centre for Computational Finance and Economic Agents (CCFEA), University of Essex, 2010.

E. P. K. Tsang. Directional changes: A new way to look at price dynamics. In *International Conference on Computational Intelligence, Communications, and Business Analytics*, pages 45–55. Springer, 2017.

E. P. K. Tsang and J. Chen. Regime change detection using directional change indicators in the foreign exchange market to chart brexit. *IEEE Transactions in Emerging Topics in Computational Intelligence*, 2(3):185–193, 2018. doi: 10.1109/TETCI.2017.2769057.

E. P. K. Tsang, R. Tao, A. Serguieva, and S. Ma. Profiling high-frequency equity price movements in directional changes. *Quantitative Finance*, 17(2):217–225, 2017. doi: 10.1080/14697688.2016.1211798.

I. Visser and M. Speekenbrink. **depmixS4**: An R package for hidden mixture models. *Journal of Statistical Software*, 36(7):1–21, 2010.

Masthead Graphic

The mast head collage presents a structured visual narrative of urban architecture, lush seasonal flora, and street-level details, split into two primary columns. The Top-Left Panel features a classic sculptural stone mask (mascaron) integrated into the building wall, a hallmark of historic Baltic architecture. The Bottom-Left Panel shows multiple tree trunks intersecting in a tree-lined boulevard setting, with sunlight filtering heavily through a thick layer of spring leaves. The Top-Right Panel shows a focused view of an upper apartment building facade featuring a striking terracotta accent wall. It highlights two symmetrical windows framed with ornate white molding and a classic wrought-iron balcony railing running across them. Green tree leaves clip into the right edge of the frame. The Middle-Right Panel displays a rich macro photograph of blooming pansies (Viola tricolor). The flowers display vivid coloration, including rich velvet red, bright golden yellow, and a distinct white pansy with a deep purple center, surrounded by soft green foliage. The Bottom-Right Panel shows a street-level architectural shot capturing a historic arched wooden door set into a light-colored stucco building. In the lower foreground, cutting across the street, the sleek, modern profile of a vibrant blue Porsche provides a stark, contemporary contrast to the historic building facade behind it.